



山东大学

计算机组成与设计

课程设计报告

报告题目：简单模型机设计（微程序和硬布线设计）

姓 名 王子瑞

学 号 202200130095

学 院 计算机科学与技术学院

专 业 计算机科学与技术专业

班 级 2022 级 4 班

2024 年 11 月 16 日

目 录

第 1 章 指令设计	1
1.1 指令格式	1
1.2 指令介绍	2
1.2.1 指令组介绍	2
1.2.2 指令详细介绍	3
1.3 指令编码	4
第 2 章 总体结构与数据通路	6
2.1 总体结构	6
2.2 数据通路	6
2.3 器件介绍	7
2.3.1 寄存器	7
2.3.2 运算器	7
2.3.3 选择器	10
2.3.4 译码器	10
2.3.5 RAM 设计	10
2.4 总体原理图	11
第 3 章 微程序实现的控制部件	12
3.1 微程序实现的 CU 框图	12
3.2 微程序控制器器件介绍	12
3.2.1 时序控制	12
3.2.2 后继地址形成电路	13
3.2.3 μ PC 设计	14
3.2.4 ROM 设计	15
3.3 指令执行流程	15
3.3.1 取指周期	15
3.3.2 执行周期	16
3.4 微指令格式	28
3.5 RAM 应用程序设计	35

3.5.1 递推计算斐波那契数列	35
3.5.2 递推计算阶乘	36
3.5.3 其他指令的混合测试	37
3.5.4 基于全局变量传参的子程序递归计算阶乘	39
3.5.5 基于栈传参的子程序递归计算阶乘	40
第4章 基于硬布线实现的模型机指令与结构的修改	43
4.1 指令格式修改	43
4.2 指令介绍	43
4.3 总体结构修改	44
4.4 RAM 修改	44
4.5 其他说明	45
4.6 总体原理图	45
第5章 硬布线实现的控制部件	46
5.1 硬布线实现的 CU 框图	46
5.2 硬布线控制器器件介绍	46
5.2.1 启动电路	46
5.2.2 节拍发生器	46
5.2.3 指令译码器	47
5.2.4 时序控制信号形成部件	48
5.3 指令执行流程	54
5.4 控制信号	54
5.4.1 控制信号列表	54
5.4.2 控制信号的逻辑表达式	54
5.5 RAM 应用程序设计	57
第6章 课程设计总结	59
6.1 遇到的问题和解决方案	59
6.2 设计的优点与缺点	60
6.2.1 设计的优点	60
6.2.2 设计的缺点	60
6.3 收获与体会	61
附录	62

附录 1: 数据通路图	62
附录 2: 微程序流程图	62
附录 3: 微程序	62
附录 4: 各控制信号的列表	62
附录 5: 各控制信号的逻辑表达式	62

第 1 章 指令设计

1.1 指令格式

本系统设计的指令字长为 16 位，共设计了三种指令格式，分别是零地址指令、一地址指令、二地址指令。通用的指令格式是操作码（5 位）、寻址方式 1（1 位）、寻址方式 2（2 位）、地址 1（4 位）、地址 2（4 位）。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
操作码 OP					寻址方式 1	寻址方式 2		操作数 1				操作数 2			

对于零地址指令来说，只采用高 5 位为操作码，后面 11 位均无意义。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
操作码 OP					/										

对于一地址指令来说，采用高 5 位为操作码，第 6 位空闲、第 7-8 位为寻址方式，这里支持立即数、直接寻址和寄存器直接寻址三种寻址方式，后面的 9-16 位为具体的操作数（立即数的值/直接寻址的地址）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
操作码 OP					/	寻址方式		操作数							

对于二地址指令来说，地址格式的设计更为灵活，采用高 5 位为操作码，第 6 位为寻址方式 1，第 7-8 位为寻址方式 2，如果是寄存器+寄存器寻址，那么第 9-10 位为寄存器 1 编号、第 13-14 位为寄存器 2 编号。如果是寄存器+立即数/地址，那么 9-10 位为寄存器编号，11-16 位为立即数/地址值，这么设计的目的是为了扩大寻址的范围和提供的操作数的大小。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
操作码 OP					寻址方式 1	寻址方式 2		寄存器 1	操作数 2						

对于寻址方式来说，01 表示立即数寻址、10 表示寄存器寻址、11 表示直接寻址、00 备用。对于寻址拓展指令（load 和 store 指令的寻址方式的拓展），01 表示单次间接寻址、10 表示 PC 相对寻址、11 表示变址寻址、00 备用。关于寄存器的编号，01 表示选中 R1 寄存器、10 表示选中 R2 寄存器、11 表示选中 R3 寄存器、00 留作备用。

1.2 指令介绍

本系统一共设计 31 类指令（31 个指令组），若考虑不同的寻址方式，能够执行约 80 条指令。不仅如此，通过不同指令的组合，可以实现更多其他的指令。比如将 SHL 指令与 LOOP 指令结合，可以实现循环左移、SHR 和 LOOP 指令结合可以实现循环右移指令。将 LOOP 指令与 JZ 或者 JNZ 指令进行组合可以实现 REP 前缀或者 REPZ 或 REPNZ 等指令。

1.2.1 指令组介绍

指令组包含了 15 条基础运算指令，其中有分为了算术运算和逻辑运算两种，如 ADD 加法指令、SUB 减法指令、MUL 乘法指令、DIV 除法指令、INC 自增 1 指令、DEC 自减指令、NEG 取反指令、AND 求与指令、OR 求或指令、XOR 异或指令、NOT 取非指令、SHL 逻辑左移指令、SAL 算数左移指令、SHR 逻辑右移指令、SAR 算术右移指令。

另外指令组中包括了 16 条其他高级指令，如 CMP 比较指令、JMP 无条件跳转指令、JZ 为零则跳转指令、JNZ 非零则跳转指令、XCHG 交换指令、LOAD 取数指令、STORE 存数指令、MOV 转移指令、LOOP 循环指令、PUSH 压栈指令、POP 弹栈指令、CALL 子程序调用指令、RET 子程序返回指令、LOADEX 取数拓展指令、STOREEX 存数拓展指令和 HLT 停机指令。

按照指令功能分类，可以看作运算类指令（15 条）、访存类指令（6 条）、跳转类指令（5 条）、栈类指令（4 条）、停机指令（1 条）。

ADD	加法指令		LOAD	取数指令
SUB	减法指令		STORE	存数指令
MUL	乘法指令		MOV	传送指令
DIV	除法指令		LOADEX	取数拓展指令
INC	自增指令		STOREEX	存数拓展指令
DEC	自减指令		XCHG	交换指令
NEG	取反指令			
AND	求与指令		CMP	比较指令
OR	求或指令		JMP	无条件跳转指令
XOR	异或指令		JZ	为零则跳转指令
NOT	取非指令		JNZ	非零则跳转指令
SHL	逻辑左移指令		LOOP	循环控制指令
SAL	算数左移指令			
SHR	逻辑右移指令		PUSH	压栈指令
SAR	算数右移指令		POP	弹栈指令
			CALL	子程序调用指令
HLT	停机指令		RET	子程序返回指令

1.2.2 指令详细介绍

对于加法、减法、乘法、除法、求与、求或、求非、求异或的操作，均为二地址指令，其中默认第一个地址为 R1 寄存器，第二个地址可以为 R2 寄存器、立即数、内存地址三种情况，并且将计算结果统一存放到第一个地址中（即 R1 中）。

对于自增、自减、取反、取非、逻辑左移、逻辑右移、算数左移、算数右移指令，均为一地址指令，由于其为一地址并且源地址为目的地址，所以其必须为寄存器，我们设计了 R1、R2、R3 三个寄存器均可以执行上面的运算。

对于 CMP 比较指令，结构与 SUB 指令相同，不过这里需要把运算结果存放到 R3 中，在本项目中，我们把 R3 寄存器视为标志寄存器和临时寄存器，即本寄存器用于特定标志的存放和数据的流转，因此在程序执行过程中，该寄存器中的值可能不是安全的（有可能在微指令中修改 R3 寄存器中的值），该寄存器的详细介绍会在下面具体介绍。

对于跳转类指令，JMP、JZ、JNZ 均为一地址指令，其中操作码为跳转的地址，而 JZ 和 JNZ 则是依据标志寄存器（R3）中的值进行判断，并且给出具体的 PC 值。

对于 XCHG 指令，这个为零地址指令，默认是将 R1 和 R2 寄存器中的值进行交换，其中需要用到 R3 寄存器暂存某个值，比如 $R1 \rightarrow R3$ ， $R2 \rightarrow R1$ ， $R3 \rightarrow R2$ ，可以看到 R3 的值出现了更改，所以会在上面说 R3 的值是不安全的，有可能会被替换掉。

对于 LOAD 和 STORE 指令来说，均是二地址指令，但是第一个地址一定是寄存器，有可能是 R1、R2 和 R3，而对于第二个地址来说，一定是内存地址。可以实现从内存中取数，放入寄存器或者将寄存器中的值放入到内存中，即 load 和 store 操作。而至于立即数或者寄存器的数据的流转，将在 MOV 指令中实现，这样根据 LOAD 和 STORE 和 MOV 指令的相互结合使用，就能够实现内存、寄存器、立即数的数据的相互轮转传递（显然不能把数据给到立即数）。

对于 MOV 指令来说，均是二地址指令，第一个地址一定寄存器，但是有可能是 R1、R2 和 R3，第二个地址有可能是立即数、也有可能是寄存器。因此在寻址方式 2 中（高 7-8 位）存放的值寻址方式，高 9-10 位一定是寻址方式 1 的寄存

器编号，如果寻址方式 2 位立即数寻址，则 11-16 位均为立即数，否则 13-14 位为寻址方式 2 的寄存器编号。

PUSH 和 POP 指令均为单地址指令，寻址方式只能为寄存器寻址，并且只能对 R1 和 R2 寄存器进行压栈，其中 00 和 11 均无效，01 和 10 分别表示选中 R1 和 R2 寄存器

CALL 为单地址指令，寻址方式为立即数寻址，给出的立即数即为子程序的开头。RET 指令为零地址指令，自动将栈顶的 PC 恢复，注意在调用 RET 指令时，需要保证子程序中没有压栈，或者所有的压栈已经全部弹栈。

LOADEX 和 STOREEX 为扩展指令，这里专门额外设计了三种寻址方式，即间接寻址、PC 相对寻址和变址寻址。其中 01 表示单次间接寻址（给出的地址为存放有效地址的地址，通过一次访存操作得到正确的地址）、10 表示 PC 相对寻址（通过 PC 和地址相加，得到真实的地址）、11 表示变址寻址，这时将 R2 寄存器看作变址寄存器，将 R2 寄存器中的值与给出的地址相加，得到真实地址。目的寄存器统一设置为 R1。

1.3 指令编码

下图详细展示了能执行的所有指令的编码方式。指令类的编号转换为二进制即为对应操作码，其中停机指令用全 1 代替，便于后续指令的扩展，全 0 的操作码备用。对于寻址方式来说，01 表示立即数寻址、10 表示寄存器寻址、11 表示直接寻址、00 备用。关于寄存器的编号，01 表示 R1、10 表示 R2、11 表示 R3、00 备用。

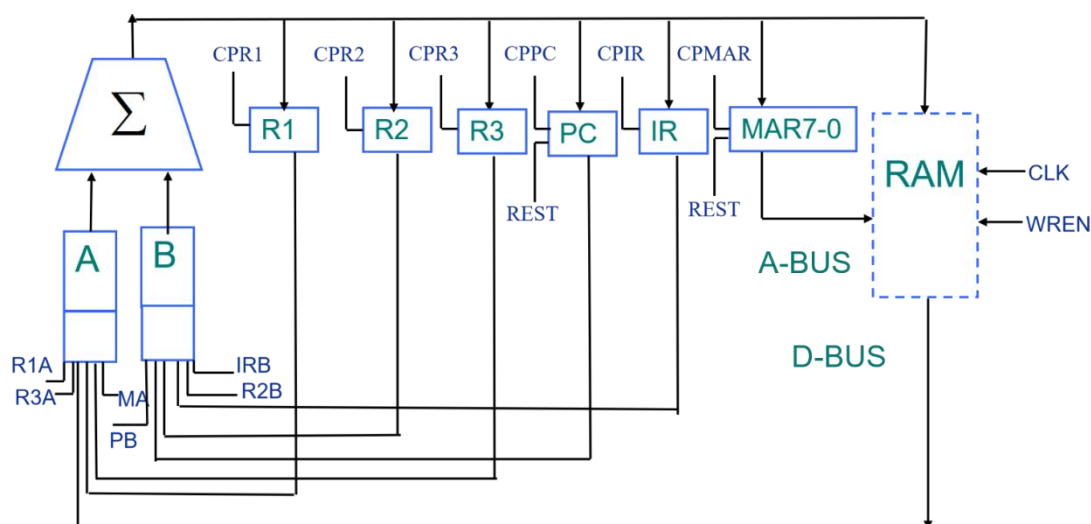
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22					
编号	指令	OP					寻址1	寻址2	数1				数2													
1	add r1 r2	0	0	0	0	1	/	1	0	/	/	/	/	/	/	/	/	/	/	/	/					
2	add r1 num							0	1	num																
3	add r1 addr							1	1	addr																
4	sub r1 r2	0	0	0	1	0	/	1	0	/	/	/	/	/	/	/	/	/	/	/	/					
5	sub r1 num							0	1	num																
6	sub r1 addr							1	1	addr																
7	mul r1 r2	0	0	0	1	1	/	1	0	/	/	/	/	/	/	/	/	/	/	/	/					
8	mul r1 num							0	1	num																
9	mul r1 addr							1	1	addr																
10	div r1 r2	0	0	1	0	0	/	1	0	/	/	/	/	/	/	/	/	/	/	/	/					
11	div r1 num							0	1	num																
12	div r1 addr							1	1	addr																
13	inc r1	0	0	1	0	0	/	0	1	num																
14	inc r2							1	1	addr																
15	inc r3							1	1																	
16	dec r1	0	0	1	1	0	/	0	1	/	/	/	/	/	/	/	/	/	/	/	/					
17	dec r2							1	0	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	
18	dec r3							1	1	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/
19	neg r1	0	0	1	1	1	/	0	1	/	/	/	/	/	/	/	/	/	/	/	/					
20	neg r2							1	0	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/
21	neg r3							1	1	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/
22																										

23	8	cmp r1 r2						1	0	/	/	/	/	/	/	/	/	/
24		cmp r1 num	0	1	0	0	0	0	1									
25		cmp r1 addr						1	1									
26		and r1 r2						1	0	/	/	/	/	/	/	/	/	/
27	9	and r1 num	0	1	0	0	1	0	1									
28		and r1 addr						1	1									
29		or r1 r2						1	0	/	/	/	/	/	/	/	/	/
30	10	or r1 num	0	1	0	1	0	0	1									
31		or r1 addr						1	1									
32		xor r1 r2						1	0	/	/	/	/	/	/	/	/	/
33	11	xor r1 num	0	1	0	1	1	0	1									
34		xor r1 addr						1	1									
35		not r1						0	1	/	/	/	/	/	/	/	/	/
36	12	not r2	0	1	1	0	0	1	0	/	/	/	/	/	/	/	/	/
37		not r3						1	1	/	/	/	/	/	/	/	/	/
38		shl r1						0	1	/	/	/	/	/	/	/	/	/
39	13	shl r2	0	1	1	0	1	1	0	/	/	/	/	/	/	/	/	/
40		shl r3						1	1	/	/	/	/	/	/	/	/	/
41		sar r1						0	1	/	/	/	/	/	/	/	/	/
42	14	sar r2	0	1	1	1	0	1	0	/	/	/	/	/	/	/	/	/
43		sar r3						1	1	/	/	/	/	/	/	/	/	/
44	15	shr r1						0	1	/	/	/	/	/	/	/	/	/
45		shr r2	0	1	1	1	1	1	0	/	/	/	/	/	/	/	/	/
46		shr r3						1	1	/	/	/	/	/	/	/	/	/
47		sar r1						0	1	/	/	/	/	/	/	/	/	/
48	16	sar r2	1	0	0	0	0	1	0	/	/	/	/	/	/	/	/	/
49		sar r3						1	1	/	/	/	/	/	/	/	/	/
50	17	jmp addr	1	0	0	0	1	/	/	/	/	/	/	/	/	/	/	/
51	18	jz addr	1	0	0	1	0	/	/	/	/	/	/	/	/	/	/	/
52	19	jnz addr	1	0	0	1	1	/	/	/	/	/	/	/	/	/	/	/
53	20	xchg	1	0	1	0	0	/	/	/	/	/	/	/	/	/	/	/
54		load r1 addr						0	1									
55	21	load r2 addr	1	0	1	0	1	/	1	0								
56		load r3 addr						1	1									
57		store r1 addr						0	1									
58	22	store r2 addr	1	0	1	1	0	/	1	0								
59		store r3 addr						1	1									
60	23	mov r1 r2	1	0	1	1	1	/	1	0	0	1	/	/	1	0	/	/
61		mov r1 r3						1	0	0	1	/	/	1	1	/	/	/
62		mov r1 num						0	1	0	1							
63		mov r2 r1						1	0	1	0	/	/	0	1	/	/	/
64		mov r2 r3	1	0	1	1	1	/	1	0	1	0	/	/	1	1	/	/
65		mov r2 num						0	1	1	0							
66		mov r3 r2						1	0	1	1	/	/	1	0	/	/	/
67		mov r3 r1	1	0	1	1	1	/	1	0	1	1	/	/	0	1	/	/
68		mov r3 num						0	1	1	1							
69	24	loop	1	1	0	0	0	/	/	/	/	/	/	/	/	/	/	/
70	25	push r1	1	1	0	0	1	/	0	1	/	/	/	/	/	/	/	/
71		push r2						1	0	/	/	/	/	/	/	/	/	/
72	26	pop r1	1	1	0	1	0	/	0	1	/	/	/	/	/	/	/	/
73		pop r2						1	0	/	/	/	/	/	/	/	/	/
74	27	call num	1	1	0	1	1	/	/	/	/	/	/	/	/	/	/	/
75	28	ret	1	1	1	0	0	/	/	/	/	/	/	/	/	/	/	/
76	29	loadEX r1 [addr]						0	1									
77		loadEX r1 [PC+addr]	1	1	1	0	1	/	1	0								
78		loadEX r1 [r2+addr]						1	1									
79	30	storeEX r1 [addr]						0	1									
80		storeEX r1 [PC+addr]	1	1	1	1	0	/	1	0								
81		storeEX r1 [r2+addr]						1	1									
82	31	hlt	1	1	1	1	1	/	/	/	/	/	/	/	/	/	/	/

第2章 总体结构与数据通路

2.1 总体结构

微程序控制实验总体结构如下：



2.2 数据通路

模型机的数据通路是以总线为基础，以 CPU 为核心构成的。

1、取指令

$$\text{RAM} \xrightarrow{\text{MA}} \text{选择器 A} \xrightarrow{\text{A 直传}} \Sigma \rightarrow \text{Bus} \xrightarrow{\text{CPIR}} \text{IR}$$

2、送指令地址

$$\text{PC} \xrightarrow{\text{PB}} \text{选择器 B} \xrightarrow{\text{B 直传}} \Sigma \rightarrow \text{Bus} \xrightarrow{\text{CPMAR}} \text{MAR}$$

3、指令计数器+1

$$\text{PC} \xrightarrow{\text{PB}} \text{选择器 B} \xrightarrow{\text{A+B+1}} \Sigma \rightarrow \text{Bus} \xrightarrow{\text{CPPC}} \text{PC}$$

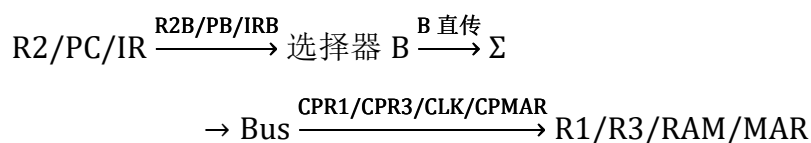
3、R1→R2

$$\text{R1} \xrightarrow{\text{R1B}} \text{选择器 A} \xrightarrow{\text{A 直传}} \Sigma \rightarrow \text{Bus} \xrightarrow{\text{CPR2}} \text{R2}$$

4、R1/R3/RAM→R2/PC/MAR

$$\text{R1/R3/RAM} \xrightarrow{\text{R1B/R3B/MB}} \text{选择器 A} \xrightarrow{\text{A 直传}} \Sigma \rightarrow \text{Bus} \xrightarrow{\text{CPR2/CPPC/CPMAR}} \text{R2/PC/MAR}$$

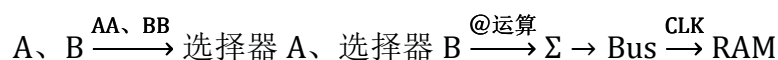
5、R2/PC/IR→R1/R3/RAM/MAR



6、A@B→R（表示任意两个寄存器进行某种运算，并且将结果放到某个寄存器）



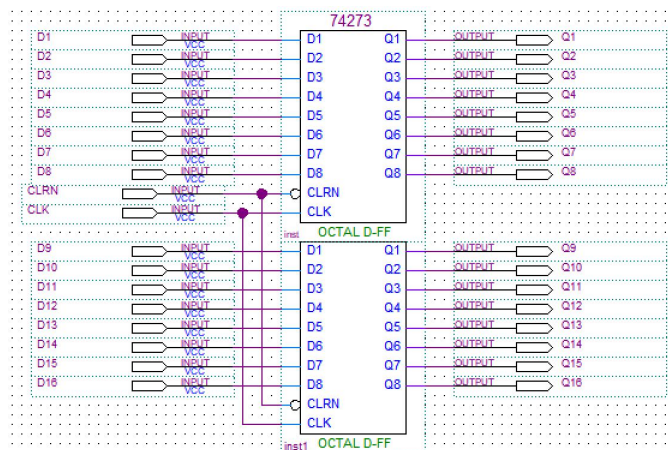
7、A@B→RAM（表示任意两个寄存器进行某种运算，并且将结果放到 RAM 中）



2.3 器件介绍

2.3.1 寄存器

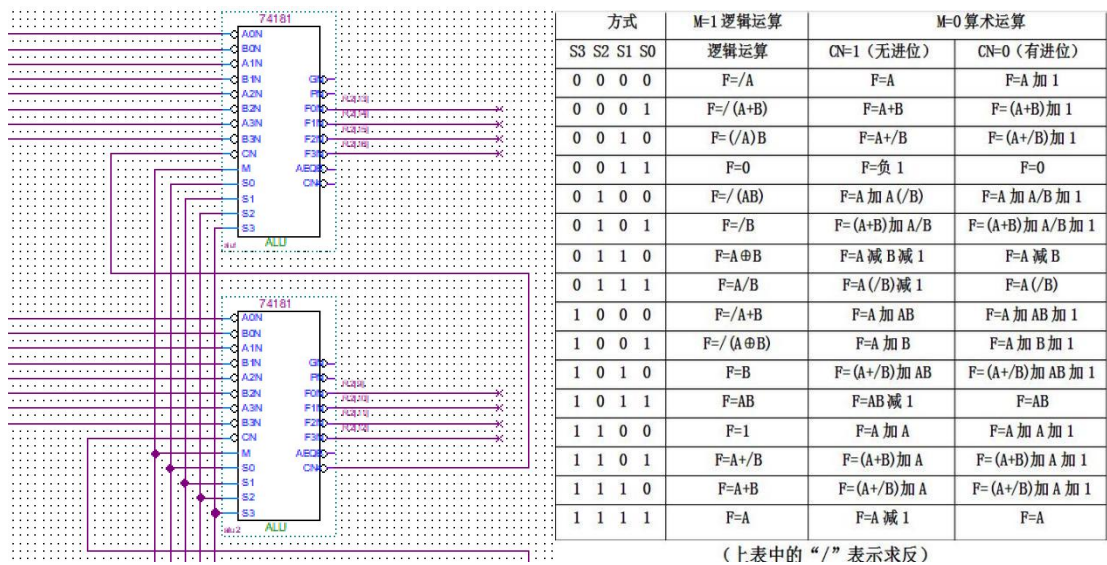
所有寄存器包括：R1 通用寄存器、R2 通用寄存器（兼变址寄存器）、R3（兼条件寄存器、临时寄存器）、MAR 地址寄存器、PC 程序计数器、IR 指令寄存器全部为 16 位。由两个 74273（8 位 D 触发器）组成。



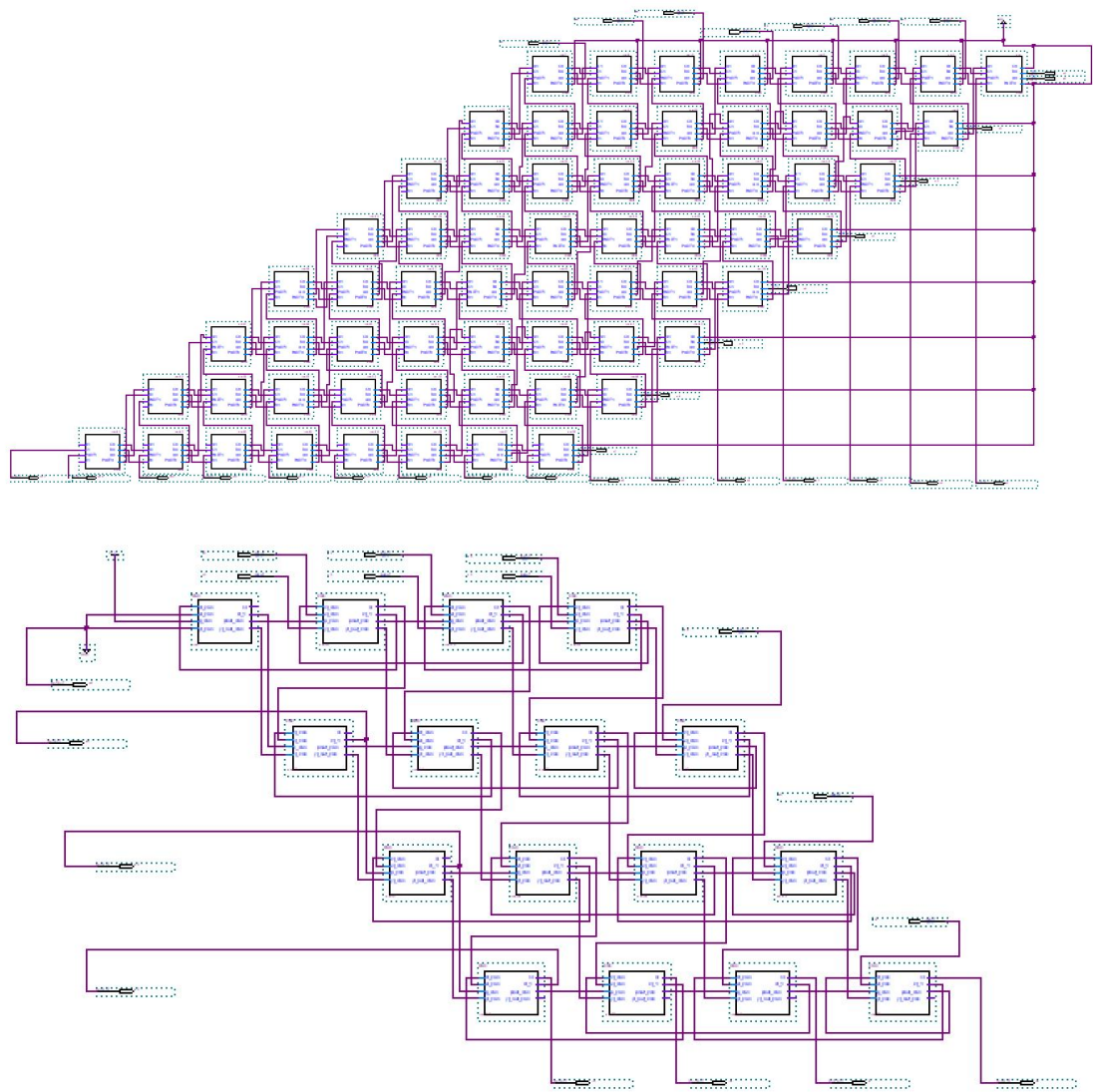
2.3.2 运算器

运算器是由基于 74181 实现的加法器以及利用了阵列乘法器、阵列除法器、基于 74198 实现的移位器和多个选择器组成的运算器 ALU。能够处理 16 位加减运算、8 位的乘法运算、除法运算和移位运算。

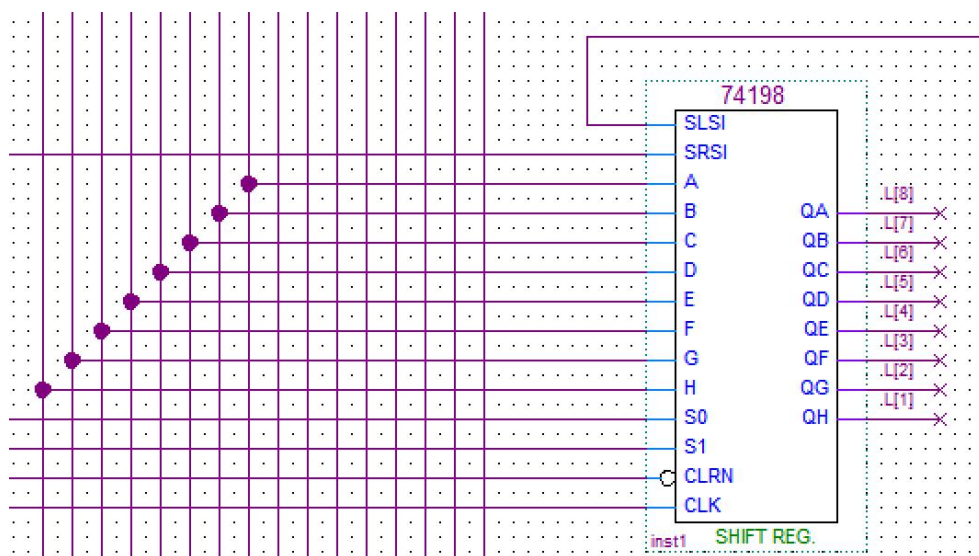
其中加法器由 4 个 74181 组成，通过 M、S3、S2、S1、CN 这五个控制信号，能够实现加法、减法、增一、减一、与、或等多种加减算数运算和多种逻辑运算。其组成如下：由于空间原因，这里只展示其中两个 74181 的原理图



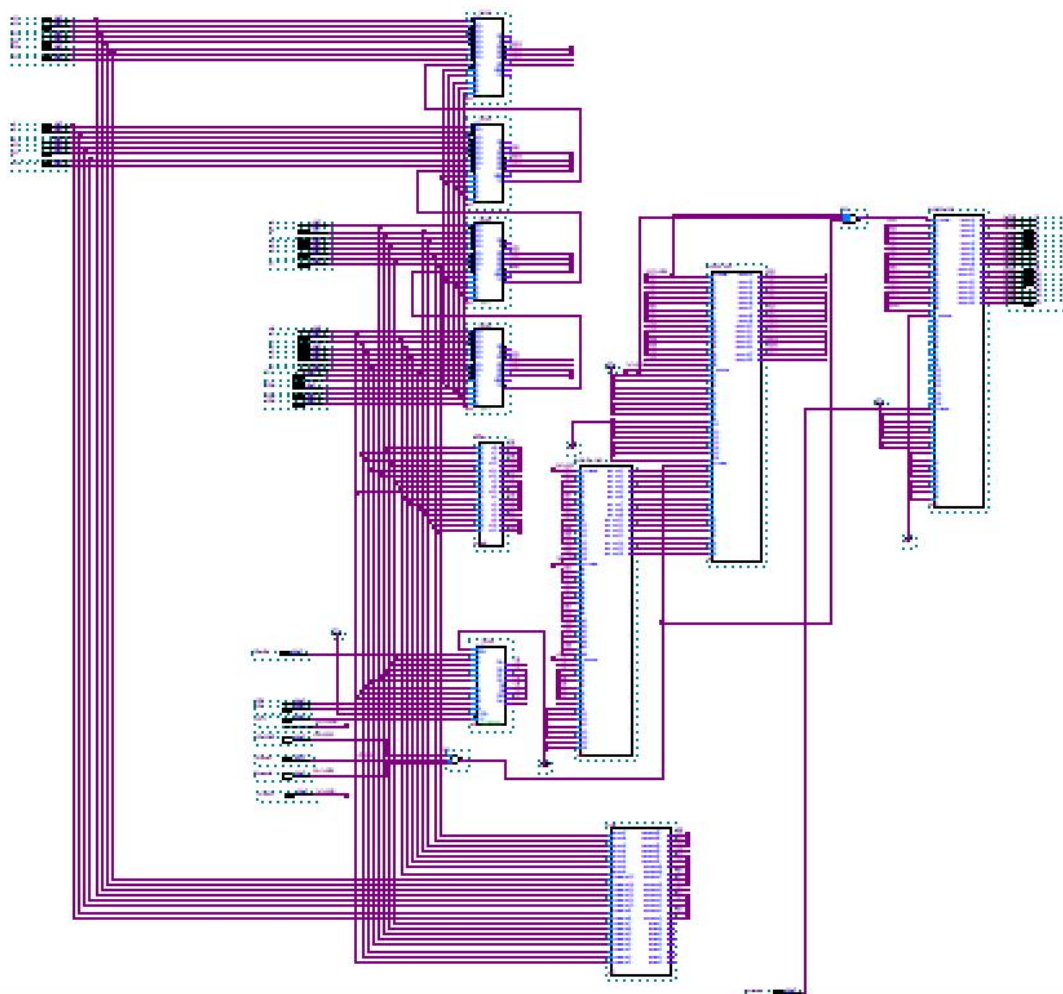
其中乘法器和除法器的实现是通过阵列乘法器和阵列除法器实现的，原理图如下：



其中移位器是通过 74198 实现的，其中的由控制信号 SLSI、SRSI、S0、S1、CLRN、CLK 这 6 个控制信号组成，控制左移或右移，并且保证其能够实现逻辑运算和算数运算两种，部分原理图如下：

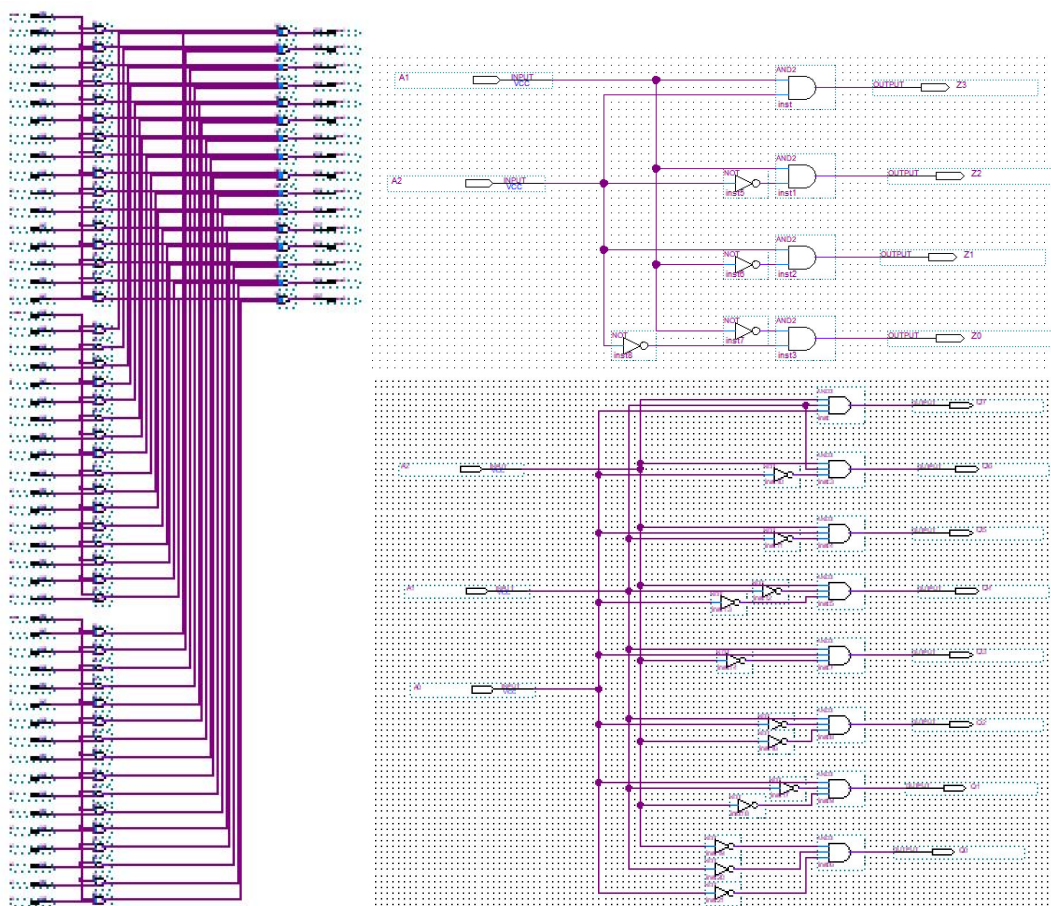


选择器作为一个独立的部件，暂时不在运算器部分中进行介绍，会在选择器部分进行详细的介绍，运算器 ALU 的原理图如下：



2.3.3 选择器

在本模型机中，一共存在 5 个具备输出信号的寄存器和 1 个 RAM，并且我们将其分配为 R1、R3、RAM 使用 A，R2、PC、IR 使用 B，因此无论是 A 或 B，都需要一个选择器来对信号进行选择，这三个选择器均为三选一选择器，因为寄存器的信号为 16 位数据，因此选择器也必须采用 16 位选择器。即 16 位三选一选择器。通过与或门就可以实现上面的逻辑。具体的选中信号的产生通过 uIR 的部分位译码后产生，在这里先不阐述，后面的 CU 部分会详细介绍。原理图如下：

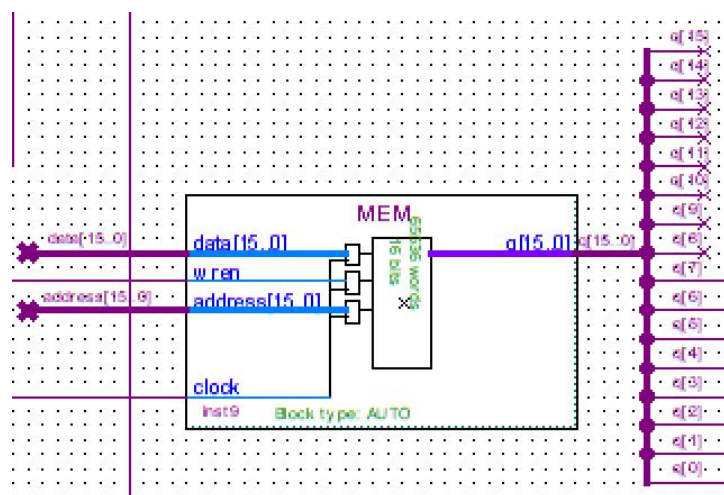


2.3.4 译码器

这里的译码器并不是对 IR 进行译码，而是对微程序控制器生成的 uIR 的部分指令进行译码，由于部分位的信号是互斥的，因此需要对其进行译码才能使用，这里用到的主要是 3-8 译码器和 2-4 译码器，通过简单的与或非门就可以实现，原理图如上所示。

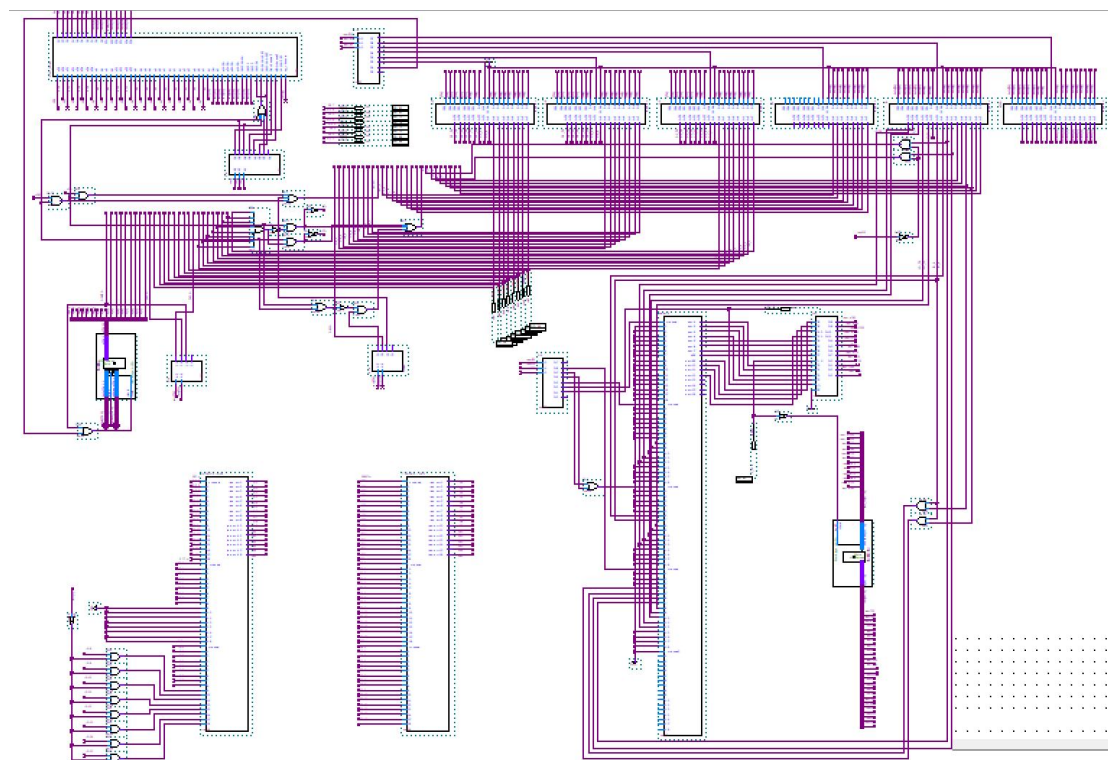
2.3.5 RAM 设计

由于我们设计的指令一共 16 位，因此 RAM 中每个单位均是 16 位，即 16bits，因为指令 16 位，假定能够访问的最大地址也是 16 位的，因此数据量设置为 $2^{16}=65536$ words，因此 RAM 的总大小设置为 65536 words * 16 bits。



2.4 总体原理图

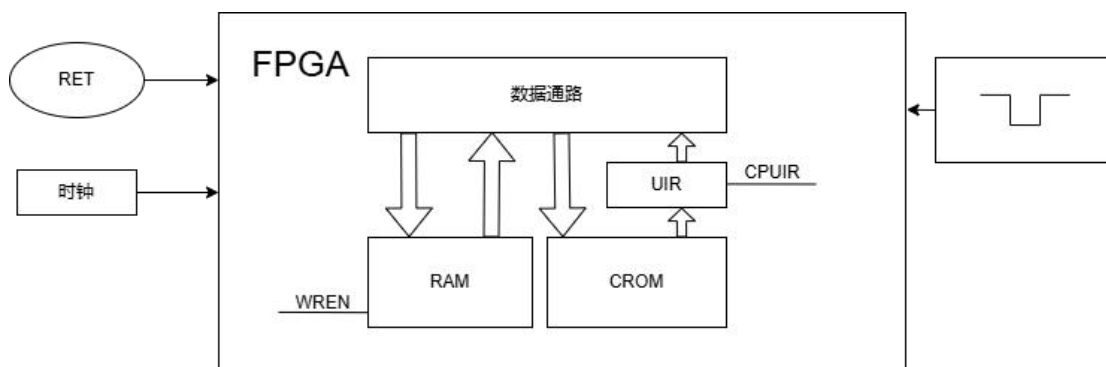
基于前面所设计的整体结构、数据通路，以及各个原件的原理图和 RAM，设计的总体原理图如下，下方右侧为微程序控制部件，将在第三章详细介绍。



第 3 章 微程序实现的控制部件

3.1 微程序实现的 CU 框图

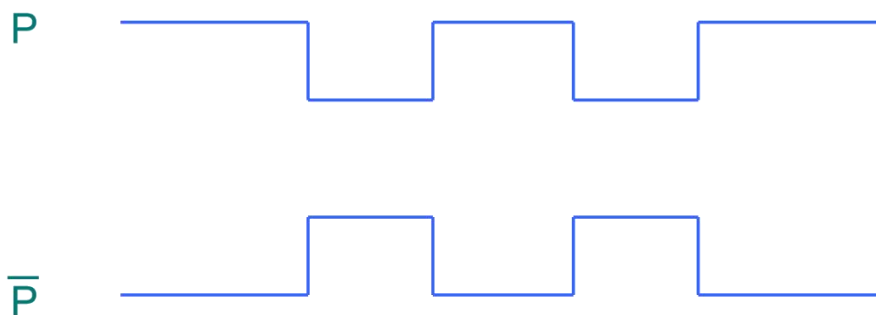
基于微程序实现的 CU 框图如下：



3.2 微程序控制器器件介绍

3.2.1 时序控制

脉冲的上升沿将形成的后继地址送给微程序计数器 uPC，而在脉冲的下降沿，将读出微指令，并且送给 uIR。并且我们一条微指令，需要通过两个 CLK 得到结果，在第一个 CLK 时，得到 uIR，并且将所有的选择信号激活，并且送到 ALU 中，在第二个 CLK 时，得到新的 uIR，该 uIR 与上一条 uIR 的区别在于本条指令将需要将输出分配激活，即将计算结果送入到对应的寄存器中，在 3.3 的指令执行流程中会详细介绍。



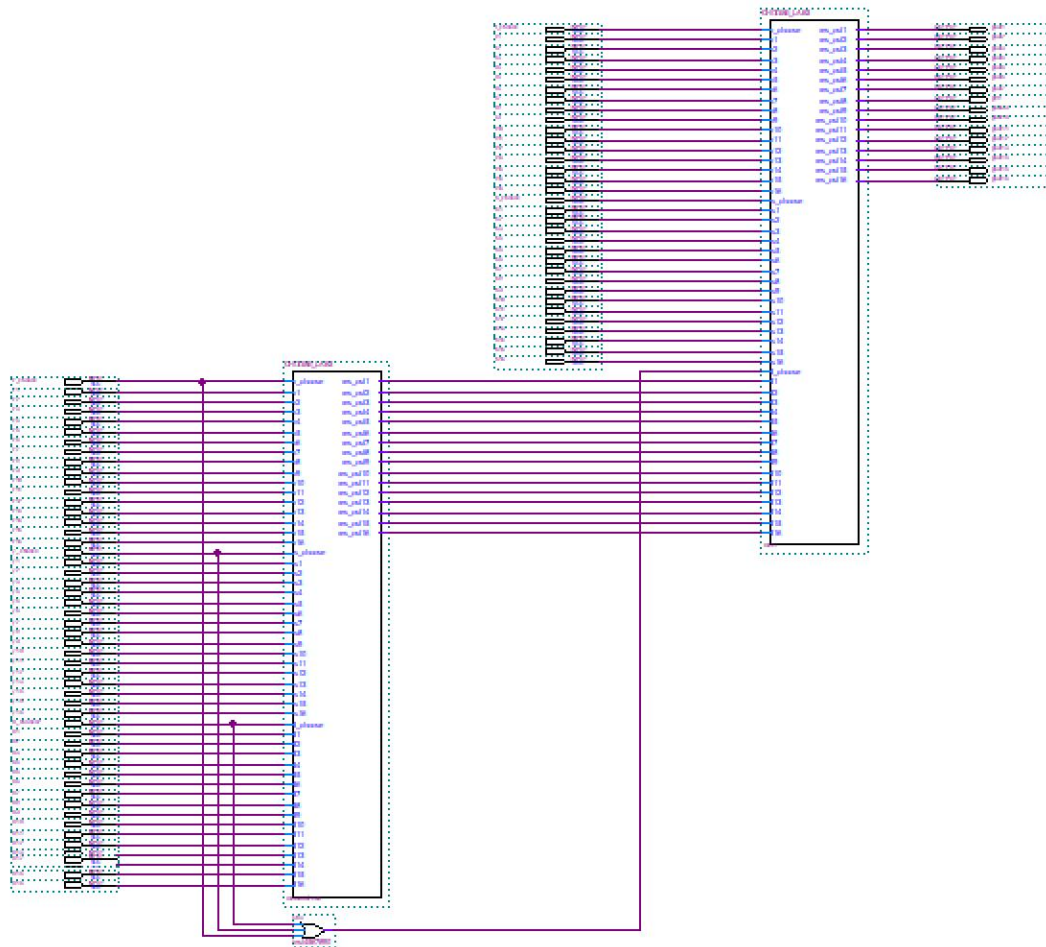
微程序控制器时序

3.2.2 后继地址形成电路

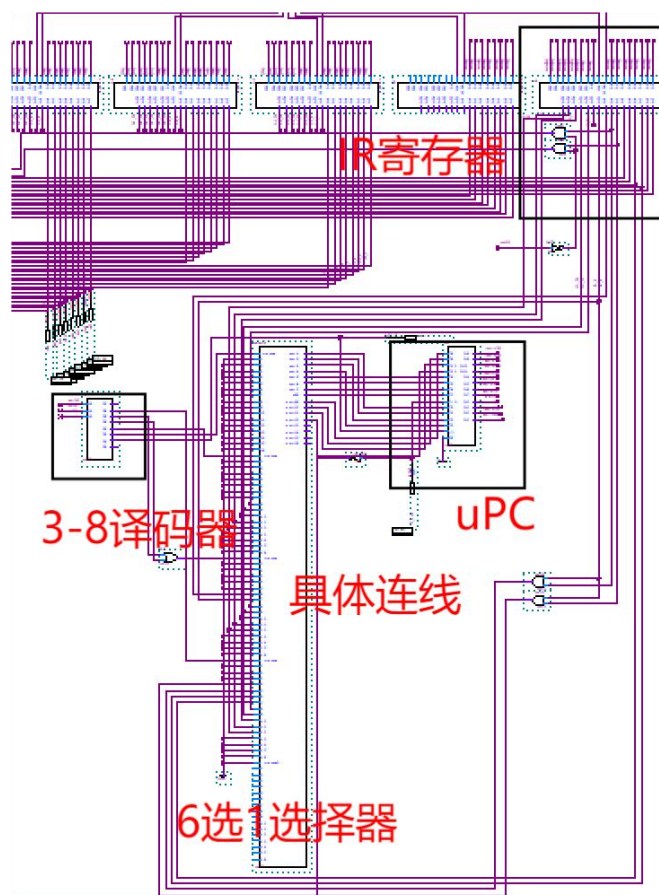
在后继地址生成中，由于需要处理分支转移，即需要根据相同指令，但是不同寻址方式，进行对应的跳转，所以这里额外设计了三种后继地址的形成方式。

uIR4	uIR3	uIR2	方式
0	0	0	备用
0	0	1	uPC+1 顺序执行
0	1	0	JP 无条件跳转 跳转到取指指令
0	1	1	QJP 操作码跳转 按照 OP 高 5 位进行跳转
1	0	0	YJP 源跳转 根据 OP 和源寻址方式进行跳转
1	0	1	MJP 目的跳转 根据 OP 和目的寻址方式进行跳转
1	1	0	MOVJP 转移跳转 为 MOV 指令设计的一种特殊跳转方式
1	1	1	备用

不难发现，主要采用了六种跳转方式，每一种跳转方式都有其独特的形成方法，需要根据指令的对应位数、跳转方式的译码所决定的，所以其后继地址形成电路本质上属于 6 选 1 选择器，由于在前面已经实现过了 3 选 1 选择器，这里利用前面实现的内容，将两个 3 选 1 选择器组合形成 6 选 1 选择器，即为我们的后继地址形成部件。

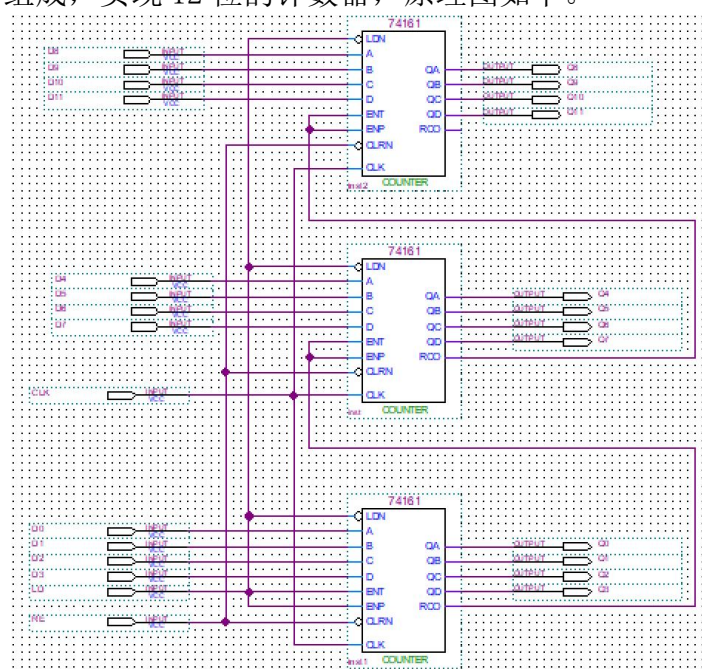


根据表格中给出的形成方式，利用 3-8 译码器和指令信号，可以构成真正的后继地址形成电路，如下：



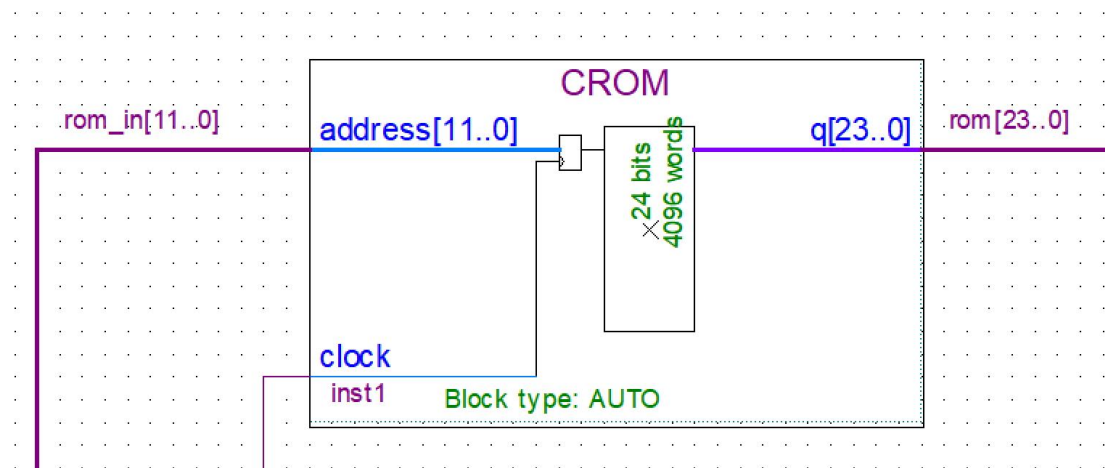
3.2.3 μ PC设计

由于微指令的地址一共 12 位，所以 μ PC 采用 3 个 74161，即三个同步四位二进制计数器组成，实现 12 位的计数器，原理图如下。



3.2.4 ROM 设计

微指令的地址一共 12 位，所以 ROM 的需要存储 2^{12} 个数据，即 4096 words，其中微指令一共 24 位，即 24 bits，所以 ROM 的大小为 4096 words * 24 bits。原理图如下：



3.3 指令执行流程

程序执行过程是在取指周期获取下一条指令，并且根据指令的 OP 进入对应指令组的微程序入口地址，然后在执行周期中，按照通过判断原操作数和目的操作数的类型进一步跳转到对应的微程序段中（即分支转移），依次执行。在执行结束后需要将 PC 放到 MAR 中，方便下一条指令的取指。

在这里由于时序原因，这里所有指令均是由两个时钟周期完成，即在第一个时钟周期仅给出对应的选择器信号，即保证选择器可以正确选择数据，对于加法器来说，可以解算出结果，并且结果在数据总线上，不过不会被打入目的寄存器或者 RAM 中。直到出现第二个 CLK，这一条指令中，我们让目的寄存器或者 RAM 的时钟信号有效，即根据前面两步，手动实现一个上跳沿，使对应的信号有效。

3.3.1 取指周期

在取指周期中只需要三步，首先根据 MAR 中的地址，从 RAM 中取指并且放到 IR 中，接下来，将 PC+1，并且放入 PC，即计算下一次执行的代码的地址。最后

根据 OP 中的值找到微程序的入口地址，这里采用的是 QJP，在后继地址生成部件中有对应的生成方式。取指过程对应的微操作如下：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	
1	指令	微操作	微地址												
2			OP				REG				NUM				
3			11-7				5-6				4	3	2	1	0
4			OP				REG				NUM				
5			11-7				3-6				2		1	0	
6			除MOV外为上面格式 MOV为下面格式												
7	取指	RAM->IR	0	0	0	0	0	0	0	0	0	0	0	0	
8			0	0	0	0	0	0	0	0	0	0	0	1	
9		0+PC+1->PC	0	0	0	0	0	0	0	0	0	0	1	0	
10			0	0	0	0	0	0	0	0	0	0	1	1	
11		QJP	0	0	0	0	0	0	0	0	0	1	0	0	

形式化地，以文字地形式描述如下：

00000（取值）-> RAM->IR -> PC+1->PC

->根据 OP 确定下一步要执行的操作

3.3.2 执行周期

对于一地址和二地址指令的执行周期需要包括两部分，首先是 YJP 或者 MJP 即根据源操作数的类别和目的操作数的类别进行一次跳转，即在该程序的操作入口，先根据细粒度的执行操作不同进行一次跳转，然后才能执行后续的操作。

先以加法 ADD 指令为例，我们给出了 ADD 共可以执行 3 种不同的操作，分别是 ADD R1,NUMBER; ADD R1,R2; ADD R1,ADDRESS，所以在进入 ADD 程序入口地址之后，先执行一次 YJP，根据源操作数的类型的不同跳转到不同的次地址中。如下图的存放微操作的微地址的 6-7 位分别为 01、10、11，而在地址 00001 01 XXX 中，执行的便是 R1+立即数的操作，并且将结果送到 R1，10 和 11 同理，不过在 R1+直接寻址的过程中，需要先进行一次访存，即需要将 IR 的对应位置先经过总线放入到 MAR 中，然后利用 MAR 中的地址在 RAM 中进行一次访存操作，然后再把访存结果放到 R2 中，最后让 R1 与 R2 进行相加，结果放到 R1，最后将 PC 送到 MAR，便于下一次取指操作，最后通过 JP 无条件跳转，返回到取指周期中。

13	ADD	YJP	0	0	0	0	1	0	0	0	0	0	0	0
14			0	0	0	0	1	0	0	0	0	0	0	0
15		R1+NUM->R1	0	0	0	0	1	0	1	0	0	0	0	0
16			0	0	0	0	1	0	1	0	0	0	0	1
17		PC->MAR	0	0	0	0	1	0	1	0	0	0	1	0
18			0	0	0	0	1	0	1	0	0	0	1	1
19		JP	0	0	0	0	1	0	1	0	0	1	0	0
20			0	0	0	0	1	0	1	0	0	0	0	0
21		R1+R2->R1	0	0	0	0	1	1	0	0	0	0	0	0
22			0	0	0	0	1	1	0	0	0	0	0	1
23		PC->MAR	0	0	0	0	1	1	0	0	0	0	1	0
24			0	0	0	0	1	1	0	0	0	0	1	1
25		JP	0	0	0	0	1	1	0	0	0	1	0	0
26			0	0	0	0	1	1	0	0	0	0	0	0
27		ADDR->MAR	0	0	0	0	1	1	1	0	0	0	0	0
28			0	0	0	0	1	1	1	0	0	0	0	1
29		RAM->R2	0	0	0	0	1	1	1	0	0	0	1	0
30			0	0	0	0	1	1	1	0	0	0	1	1
31		R1+R2->R1	0	0	0	0	1	1	1	0	0	1	0	0
32			0	0	0	0	1	1	1	0	0	1	0	1
33		PC->MAR	0	0	0	0	1	1	1	0	0	1	1	0
34			0	0	0	0	1	1	1	0	0	1	1	1
35		JP	0	0	0	0	1	1	1	0	1	0	0	0

减法、乘法、除法、以及其他双操作数的运算均与加法相似，由于篇幅原因，下面以文字形式进行展示：

（加法）

0000100→ YJP（根据源操作数类型进行跳转）

0000101（寄存器立即数）→ $R1+NUM \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0000110（寄存器寄存器）→ $R1+R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0000111（寄存器存储器）→ $ADDR \rightarrow MAR$ → $RAM \rightarrow R2$
→ $R1+R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

（减法）

0001000→ YJP（根据源操作数类型进行跳转）

0001001（寄存器立即数）→ $R1-NUM \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0001010（寄存器寄存器）→ $R1-R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0001011（寄存器存储器）→ $ADDR \rightarrow MAR$ → $RAM \rightarrow R2$
→ $R1-R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

（乘法）

0001100→ YJP（根据源操作数类型进行跳转）

0001101（寄存器立即数）→ $R1*NUM \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0001110（寄存器寄存器）→ $R1*R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0001111（寄存器存储器）→ $ADDR \rightarrow MAR$ → $RAM \rightarrow R2$
→ $R1*R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

（除法）

0010000→ YJP（根据源操作数类型进行跳转）

0010001（寄存器立即数）→ $R1/NUM \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0010010（寄存器寄存器）→ $R1/R2 \rightarrow R1$ → $PC \rightarrow MAR$ → JP

0010011（寄存器存储器）→ $ADDR \rightarrow MAR$ → $RAM \rightarrow R2$

-> R1/R2->R1 -> PC->MAR -> JP

(与)

0100100-> YJP (根据源操作数类型进行跳转)

0100101 (寄存器立即数) -> R1&NUM->R1 -> PC->MAR -> JP

0100110 (寄存器寄存器) -> R1&R2->R1 -> PC->MAR -> JP

0100111 (寄存器存储器) -> ADDR->MAR -> RAM->R2
-> R1&R2->R1 -> PC->MAR -> JP

(或)

0101000-> YJP (根据源操作数类型进行跳转)

0101001 (寄存器立即数) -> R1|NUM->R1 -> PC->MAR -> JP

0101010 (寄存器寄存器) -> R1|R2->R1 -> PC->MAR -> JP

0101011 (寄存器存储器) -> ADDR->MAR -> RAM->R2
-> R1|R2->R1 -> PC->MAR -> JP

(异或)

0101100-> YJP (根据源操作数类型进行跳转)

0101101 (寄存器立即数) -> R1^NUM->R1 -> PC->MAR -> JP

0101110 (寄存器寄存器) -> R1^R2->R1 -> PC->MAR -> JP

0101111 (寄存器存储器) -> ADDR->MAR -> RAM->R2
-> R1^R2->R1 -> PC->MAR -> JP

(比较)

0100000-> YJP (根据原操作进行跳转)

0100001 (寄存器立即数) -> R1-NUM->R3 -> PC->MAR -> JP

0100010 (寄存器寄存器) -> R1-R2->R3 -> PC->MAR -> JP

0100011 (寄存器存储器) -> ADDR->MAR -> RAM->R2
-> R1-R2->R3 -> PC->MAR -> JP

我们再以单操作数的运算进行介绍。先以最简单的 DEC 指令为例，由于器件只能执行 $A+B$ 、 $A+1$ 、 $A+B+1$ 、 $A-B$ 、 $A-1$ 的操作，因此无法不能直接执行 $B-1$ 的操作，并不能直接执行 $R2-1 \rightarrow R2$ 的操作（RAM R1 R3 在 A 中，R2 PC IR 在 B 中），这时有两种解决方案，第一种是参考 $PC+1 \rightarrow PC$ 的过程，执行 $A+B+1$ 的操作，注意这里 A 给的值是 0，因此变相执行了 $B+1$ 的操作，这种方法可以在 INC，即自增 1 中实现，但是减法仍然不能直接完成。我们这里采用先把 R2 利用直送操作送到 R3 中，然后再执行 $R3+1 \rightarrow R2$ 的操作，利用好 R3 寄存器，便可以解决上面的问题。这种解决问题的方法在 NEG、INC、NOT、SHL、SAL、SHR、SAR 等指令中都有所体现。

131	DEC	YJP	0	0	1	1	0	0	0	0	0	0	0	0	0
132			0	0	1	1	0	0	1	0	0	0	0	0	0
133		R1-1->R1	0	0	1	1	0	0	1	0	0	0	0	0	0
134			0	0	1	1	0	0	1	0	0	0	0	0	1
135		PC->MAR	0	0	1	1	0	0	1	0	0	0	1	0	0
136			0	0	1	1	0	0	1	0	0	0	1	1	0
137		JP	0	0	1	1	0	0	1	0	0	1	0	0	0
138			0	0	1	1	0	0	1	0	0	0	0	0	0
139			0	0	1	1	0	1	0	0	0	0	0	0	0
140		R2->R3	0	0	1	1	0	1	0	0	0	0	0	0	1
141			0	0	1	1	0	1	0	0	0	0	1	0	0
142		R3-1->R2	0	0	1	1	0	1	0	0	0	0	1	1	0
143			0	0	1	1	0	1	0	0	0	1	0	0	0
144		PC->MAR	0	0	1	1	0	1	0	0	0	1	0	0	0
145			0	0	1	1	0	1	0	0	0	1	0	1	0
146		JP	0	0	1	1	0	1	0	0	0	1	1	0	0
147	DEC		0	0	1	1	0	1	1	0	0	0	0	0	0
148		R3-1->R3	0	0	1	1	0	1	1	0	0	0	0	0	1
149			0	0	1	1	0	1	1	0	0	0	1	0	0
150		PC->MAR	0	0	1	1	0	1	1	0	0	0	1	1	0
151			0	0	1	1	0	1	1	0	0	1	0	0	0
151		JP	0	0	1	1	0	1	1	0	0	1	0	0	0

其他相似指令的流程如下：

（自增）

0010100-> YJP（根据操作寄存器进行跳转）

0010101-> R1+1->R1 -> PC->MAR -> JP

0010110-> R2->R3 -> R3+1->R2 -> PC->MAR -> JP

0010111-> R3+1->R3 -> PC->MAR -> JP

（自减）

0011000-> YJP（根据操作寄存器进行跳转）

0011001-> R1-1->R1 -> PC->MAR -> JP

0011010-> R2->R3 -> R3-1->R2 -> PC->MAR -> JP

0011011-> R3-1->R3 -> PC->MAR -> JP

（取反）

0011100→ YJP (根据操作寄存器进行跳转)

0011101→ 0-R1→R1 → PC→MAR → JP

0011110→ R2→R3 → 0-R3→R2 → PC→MAR → JP

0011111→ 0-R3→R3 → PC→MAR → JP

(非)

0110000→ YJP (根据操作寄存器进行跳转)

0110001→ $\neg$ R1→R1 → PC→MAR → JP

0110010→ $\neg$ R2→R2 → PC→MAR → JP

(逻辑左移)

0110100→ YJP (根据操作寄存器进行跳转)

0110101→ R1<<1 →R1 → PC→MAR → JP

0110110→ R2→R3 → R3<<1 →R2 → PC→MAR → JP

0110111→ R3<<1 →R3 → PC→MAR → JP

(算数左移)

0111000→ YJP (根据操作寄存器进行跳转)

0111001→ R1<<1 →R1 → PC→MAR → JP

0111010→ R2→R3 → R3<<1 →R2 → PC→MAR → JP

0111011→ R3<<1 →R3 → PC→MAR → JP

(逻辑右移)

0111100→ YJP (根据操作寄存器进行跳转)

0111101→ R1>>1 →R1 → PC→MAR → JP

0111110→ R2→R3 → R3>>1 →R2 → PC→MAR → JP

0111111-> R3>>1 ->R3 -> PC->MAR -> JP

(算数右移)

1000000-> YJP (根据操作寄存器进行跳转)

1000001-> R1>>1 ->R1 -> PC->MAR -> JP

1000010-> R2->R3 -> R3>>1 ->R2 -> PC->MAR -> JP

1000011-> R3>>1 ->R3 -> PC->MAR -> JP

接下来我们考虑跳转指令，无条件跳转指令是最简单的，由于跳转的目的地址已经在指令中给出，因此直接送到 IR 中，而 JZ 和 JNZ 指令则更为复杂，由于判断条件处于 R3 中，因此需要将 R3 选中，并且根据 JZ 和 JNZ 的特有信号产生是选中 IR 还是选中 PC 的信号，并且将其送到 PC，最后让 PC 送到 IR。具体的实现过程在数据选择器中会有详细的介绍。而至于交换指令，只是利用 R3 作为临时寄存器保存 R1 中的值，然后将 R2 赋值给 R1，然后 R3 赋值给 R2，实现了数据的交换，该操作与 C/C++ 中的 swap 操作一致。

405	JMP	IR->PC	1	0	0	0	1	0	0	0	0	0	0	0	0	0
406			1	0	0	0	1	0	0	0	0	0	0	0	1	
407		PC->MAR	1	0	0	0	1	0	0	0	0	0	0	1	0	
408			1	0	0	0	1	0	0	0	0	0	0	1	1	
409		JP	1	0	0	0	1	0	0	0	0	0	1	0	0	
410																
411	JZ	(IF0)IR->PC	1	0	0	1	0	0	0	0	0	0	0	0	0	
412			1	0	0	1	0	0	0	0	0	0	0	0	1	
413		PC->MAR	1	0	0	1	0	0	0	0	0	0	0	1	0	
414			1	0	0	1	0	0	0	0	0	0	0	1	1	
415		JP	1	0	0	1	0	0	0	0	0	0	1	0	0	

417	JNZ	(IF^0)IR->PC	1	0	0	1	1	0	0	0	0	0	0	0	0	
418			1	0	0	1	1	0	0	0	0	0	0	0	1	
419		PC->MAR	1	0	0	1	1	0	0	0	0	0	0	1	0	
420			1	0	0	1	1	0	0	0	0	0	0	1	1	
421		JP	1	0	0	1	1	0	0	0	0	0	1	0	0	
422																
423	XCHG	R1->R3	1	0	1	0	0	0	0	0	0	0	0	0	0	
424			1	0	1	0	0	0	0	0	0	0	0	0	1	
425		R2->R1	1	0	1	0	0	0	0	0	0	0	0	1	0	
426			1	0	1	0	0	0	0	0	0	0	0	1	1	
427		R3->R2	1	0	1	0	0	0	0	0	0	0	1	0	0	
428			1	0	1	0	0	0	0	0	0	0	1	0	1	
429		PC->MAR	1	0	1	0	0	0	0	0	0	0	1	1	0	
430			1	0	1	0	0	0	0	0	0	0	1	1	1	
431		JP	1	0	1	0	0	0	0	0	0	1	0	0	0	

形式化的表达如下：

(JMP)

10001 -> IR->PC -> PC->MAR -> JP

(JZ)

10010 -> IR->PC -> PC->MAR -> JP

(JNZ)

10011 -> IR->PC -> PC->MAR -> JP

(XCHG)

10100 -> R1->R3 -> R2->R1 -> R3->R2 -> PC->MAR -> JP

在 LOAD 和 STORE 指令中，必定会涉及到内存中的单元，因此需要先将 IR 送到 MAR，然后根据指令 MJP 判断选择的寄存器是哪一个，然后再执行 RAM->寄存器或者寄存器->RAM 的操作。如下左图。

433		IR->MAR	1	0	1	0	1	0	0	0	0	0	0	0
434			1	0	1	0	1	0	0	0	0	0	0	1
435		MJP	1	0	1	0	1	0	0	0	0	0	1	0
436														
437		RAM->R1	1	0	1	0	1	0	1	0	0	0	0	0
438			1	0	1	0	1	0	1	0	0	0	0	1
439		PC->MAR	1	0	1	0	1	0	1	0	0	0	1	0
440			1	0	1	0	1	0	1	0	0	0	1	1
441		JP	1	0	1	0	1	0	1	0	0	1	0	0
442														
443	LOAD	RAM->R2	1	0	1	0	1	1	0	0	0	0	0	0
444			1	0	1	0	1	1	0	0	0	0	0	1
445		PC->MAR	1	0	1	0	1	1	0	0	0	0	1	0
446			1	0	1	0	1	1	0	0	0	0	1	1
447		JP	1	0	1	0	1	1	0	0	0	1	0	0
448														
449		RAM->R3	1	0	1	0	1	1	1	0	0	0	0	0
450			1	0	1	0	1	1	1	0	0	0	0	1
451		PC->MAR	1	0	1	0	1	1	1	0	0	0	1	0
452			1	0	1	0	1	1	1	0	0	0	1	1
453		JP	1	0	1	0	1	1	1	0	0	1	0	0
455		IR->MAR	1	0	1	1	0	0	0	0	0	0	0	0
456			1	0	1	1	0	0	0	0	0	0	0	1
457		MJP	1	0	1	1	0	0	0	0	0	0	1	0
458														
459		R1->RAM	1	0	1	1	0	0	1	0	0	0	0	0
460			1	0	1	1	0	0	1	0	0	0	0	1
461		PC->MAR	1	0	1	1	0	0	1	0	0	0	1	0
462			1	0	1	1	0	0	1	0	0	0	1	1
463		JP	1	0	1	1	0	0	1	0	0	1	0	0
464														
465	STORE	R2->RAM	1	0	1	1	0	1	0	0	0	0	0	0
466			1	0	1	1	0	1	0	0	0	0	0	1
467		PC->MAR	1	0	1	1	0	1	0	0	0	0	1	0
468			1	0	1	1	0	1	0	0	0	0	1	1
469		JP	1	0	1	1	0	1	0	0	0	1	0	0
470														
471		R3->RAM	1	0	1	1	0	1	1	0	0	0	0	0
472			1	0	1	1	0	1	1	0	0	0	0	1
473		PC->MAR	1	0	1	1	0	1	1	0	0	0	1	0
474			1	0	1	1	0	1	1	0	0	0	1	1
475		JP	1	0	1	1	0	1	1	0	0	1	0	0

至于 LOOP 指令,这里我们将专门存放循环变量的 CX 寄存器设置在 RAM 的全 1 位置,因此第一步的操作就是先将全 1 的地址,送到 MAR 中,然后将 RAM 中的数值送到 R3 中,并且实现 R3 的-1 操作,接下来还需要将该值送回到 RAM 中,接下来需要判断-1 之后的结果是否为 0,如果为 0 则跳出循环,即执行 PC+1 的操作,否则需要重新跳转到 LOOP 指令指定的地址中,所以这里对应的恰好是非零则跳转操作,最后将 PC 送到 MAR 中,然后重新取指。

533		SPE->MAR	1	1	0	0	0	0	0	0	0	0	0	0	0
534			1	1	0	0	0	0	0	0	0	0	0	0	1
535		RAM->R3	1	1	0	0	0	0	0	0	0	0	0	1	0
536			1	1	0	0	0	0	0	0	0	0	0	1	1
537		R3-1->R3	1	1	0	0	0	0	0	0	0	0	0	1	0
538			1	1	0	0	0	0	0	0	0	0	0	1	0
539		SPE->MAR	1	1	0	0	0	0	0	0	0	0	1	1	0
540	LOOP		1	1	0	0	0	0	0	0	0	0	1	1	1
541		R3->RAM	1	1	0	0	0	0	0	0	0	1	0	0	0
542			1	1	0	0	0	0	0	0	0	1	0	0	1
543		(IF^0)IR->PC	1	1	0	0	0	0	0	0	0	1	0	1	0
544			1	1	0	0	0	0	0	0	0	1	0	1	1
545		PC->MAR	1	1	0	0	0	0	0	0	0	1	1	0	0
546			1	1	0	0	0	0	0	0	0	1	1	0	1
547		JP	1	1	0	0	0	0	0	0	0	1	1	1	0

MOV 指令与 LOAD 和 STORE 指令非常相似,不过只设计寄存器和寄存器、寄存器和立即数的转移操作,因此这里的跳转方式与前面的 JYP 和 MJP 均不同,特意为 MOV 操作设计了一种跳转方式 MOVJP,这种跳转方式会根据操作数 1 和操作数 2 的不同类型跳转到不同的微地址中,便于其他指令的执行。如下:

477		MOVJP	1	0	1	1	1	0	0	0	0	0	0	0	0
478			1	0	1	1	1	0	0	0	0	0	0	0	0
479		IR->R1	1	0	1	1	1	0	1	0	0	0	0	0	1
480			1	0	1	1	1	0	1	0	0	0	0	0	1
481		PC->MAR	1	0	1	1	1	0	1	0	0	0	1	0	0
482			1	0	1	1	1	0	1	0	0	0	1	1	0
483		JP	1	0	1	1	1	0	1	0	0	1	0	0	0
484			1	0	1	1	1	0	1	0	0	1	0	0	0
485		R2->R1	1	0	1	1	1	0	1	1	0	0	0	0	0
486			1	0	1	1	1	0	1	1	0	0	0	0	1
487		PC->MAR	1	0	1	1	1	0	1	1	0	0	1	0	0
488			1	0	1	1	1	0	1	1	0	0	1	1	0
489		JP	1	0	1	1	1	0	1	1	0	1	0	0	0
490			1	0	1	1	1	0	1	1	0	1	0	0	0
491		R3->R1	1	0	1	1	1	0	1	1	1	0	0	0	0
492			1	0	1	1	1	0	1	1	1	0	0	0	1
493		PC->MAR	1	0	1	1	1	0	1	1	1	0	1	0	0
494			1	0	1	1	1	0	1	1	1	0	1	1	0
495		JP	1	0	1	1	1	0	1	1	1	1	0	0	0
496			1	0	1	1	1	0	1	1	1	1	0	0	0
497		IR->R2	1	0	1	1	1	1	0	0	0	0	0	0	0
498			1	0	1	1	1	1	0	0	0	0	0	0	1
499		PC->MAR	1	0	1	1	1	1	0	0	0	0	1	0	0
500			1	0	1	1	1	1	0	0	0	0	1	1	0
501		JP	1	0	1	1	1	1	0	0	0	1	0	0	0
502			1	0	1	1	1	1	0	0	0	1	0	0	0
503		R1->R2	1	0	1	1	1	1	0	0	1	0	0	0	0
504	MOV		1	0	1	1	1	1	0	0	1	0	0	0	1
505		PC->MAR	1	0	1	1	1	1	0	0	1	0	1	0	0
506			1	0	1	1	1	1	0	0	1	0	1	1	0
507		JP	1	0	1	1	1	1	0	0	1	1	0	0	0

508													
509	R3->R2	1	0	1	1	1	1	0	1	1	0	0	0
510		1	0	1	1	1	1	0	1	1	0	0	1
511	PC->MAR	1	0	1	1	1	1	0	1	1	0	1	0
512		1	0	1	1	1	1	0	1	1	0	1	1
513	JP	1	0	1	1	1	1	0	1	1	1	0	0
514													
515	IR->R3	1	0	1	1	1	1	1	0	0	0	0	0
516		1	0	1	1	1	1	1	0	0	0	0	1
517	PC->MAR	1	0	1	1	1	1	1	0	0	0	1	0
518		1	0	1	1	1	1	1	0	0	0	1	1
519	JP	1	0	1	1	1	1	1	0	0	1	0	0
520													
521	R1->R3	1	0	1	1	1	1	1	0	1	0	0	0
522		1	0	1	1	1	1	1	0	1	0	0	1
523	PC->MAR	1	0	1	1	1	1	1	0	1	0	1	0
524		1	0	1	1	1	1	1	0	1	0	1	1
525	JP	1	0	1	1	1	1	1	0	1	1	0	0
526													
527	R2->R3	1	0	1	1	1	1	1	1	0	0	0	0
528		1	0	1	1	1	1	1	1	0	0	0	1
529	PC->MAR	1	0	1	1	1	1	1	1	0	0	1	0
530		1	0	1	1	1	1	1	1	0	0	1	1
531	JP	1	0	1	1	1	1	1	1	0	1	0	0

接下来介绍栈相关的指令，首先给出栈的相关设置，栈顶指针为 sp，由于寄存器的限制，我们将该指针放到内存中，其内存地址为 0000 0000 0111 1111，并且我们将栈底设置为 0000 0000 1111 1111，并且采用的是递减满栈的形式，即 push 指令是让栈顶上移动，并且保证 sp 指向的地方是有值的。

关于 push 和 pop 指令，先介绍 push 指令，由于是递减满栈，所以需要先从内存中得到 sp 指向的内容，即现在的栈顶，然后将其-1，并且重新写入内存（修改 sp 指针指向的位置），然后将-1 后的值送入 MAR，即需要需要向栈中写入对应的内容。下面经过一条 YJP，根据源操作数的类型，来判断是将 R1 寄存器还是 R2 寄存器中的值放入栈中。

549													
550	SPE2->MAR	1	1	0	0	1	0	0	0	0	0	0	0
551		1	1	0	0	1	0	0	0	0	0	0	1
552	RAM->R3	1	1	0	0	1	0	0	0	0	0	1	0
553		1	1	0	0	1	0	0	0	0	0	1	1
554	R3-1->RAM	1	1	0	0	1	0	0	0	0	1	0	0
555		1	1	0	0	1	0	0	0	0	1	0	1
556	R3-1->MAR	1	1	0	0	1	0	0	0	0	1	1	0
557		1	1	0	0	1	0	0	0	0	1	1	1
558	YJP	1	1	0	0	1	0	0	0	1	0	0	0
559													
560	R1->RAM	1	1	0	0	1	0	1	0	0	0	0	0
561		1	1	0	0	1	0	1	0	0	0	0	1
562	PC->MAR	1	1	0	0	1	0	1	0	0	0	1	0
563		1	1	0	0	1	0	1	0	0	0	1	1
564	JP	1	1	0	0	1	0	1	0	0	1	0	0
565													
566	R2->RAM	1	1	0	0	1	1	0	0	0	0	0	0
567		1	1	0	0	1	1	0	0	0	0	0	1
568	PC->MAR	1	1	0	0	1	1	0	0	0	0	1	0
569		1	1	0	0	1	1	0	0	0	0	1	1
	JP	1	1	0	0	1	1	0	0	0	1	0	0

pop 指令恰好与 push 指令相反，即这里需要将 sp 指针+1，并且根据 YJP，将栈中的内容恢复给 R1 或者 R2 寄存器。

571	POP	SPE2->MAR	1	1	0	1	0	0	0	0	0	0	0	0	0
572			1	1	0	1	0	0	0	0	0	0	0	0	1
573		RAM->R3	1	1	0	1	0	0	0	0	0	0	1	0	0
574			1	1	0	1	0	0	0	0	0	0	1	1	0
575		R3+1>RAM	1	1	0	1	0	0	0	0	0	1	0	0	0
576			1	1	0	1	0	0	0	0	0	1	0	1	0
577		R3->MAR	1	1	0	1	0	0	0	0	0	1	1	0	0
578			1	1	0	1	0	0	0	0	0	1	1	1	0
579		YJP	1	1	0	1	0	0	0	0	1	0	0	0	0
580															
581		RAM->R1	1	1	0	1	0	0	1	0	0	0	0	0	0
582			1	1	0	1	0	0	1	0	0	0	0	0	1
583		PC->MAR	1	1	0	1	0	0	1	0	0	0	1	0	0
584			1	1	0	1	0	0	1	0	0	0	1	1	1
585		JP	1	1	0	1	0	0	1	0	0	1	0	0	0
586															
587	POP	RAM->R2	1	1	0	1	0	1	0	0	0	0	0	0	0
588			1	1	0	1	0	1	0	0	0	0	0	0	1
589		PC->MAR	1	1	0	1	0	1	0	0	0	0	1	0	0
590			1	1	0	1	0	1	0	0	0	0	1	1	1
591		JP	1	1	0	1	0	1	0	0	0	1	0	0	0
591															

接下来考虑栈的进阶操作，即子程序调用，首先介绍子程序调用的传参方法，这里允许采用全局变量传参，或者利用栈传参，栈传参需要用到上面的 push 和 pop 指令，但是只要是采用了子程序调用，就必须使用下面介绍的 call 和 ret 指令。其中 call 指令的一个重要作用就是保护原本的 PC，并且将调用的程序的地址放入 PC 中，而 ret 指令也恰好相反，是向栈中的 PC 恢复。不过需要注意的是，在子程序调用过程中，一定要保证子程序中所有 push 的内容，全部被 pop，否则调用 ret 指令时，返回值的内容将出现错误，即 PC 无法正常恢复。

其中 call 指令与 push 的区别在于 push 指令是将 R1 或者 R2 寄存器中的值放入栈中，而 call 指令是将 PC 的值放入栈中。而 ret 指令与 pop 指令的区别是 pop 指令是将 R1 或者 R2 的值从栈中取出并且恢复，而 ret 指令是将 PC 的值进行恢复。

593	CALL	SPE2->MAR	1	1	0	1	1	0	0	0	0	0	0	0	0
594			1	1	0	1	1	0	0	0	0	0	0	0	1
595		RAM->R3	1	1	0	1	1	0	0	0	0	0	1	0	0
596			1	1	0	1	1	0	0	0	0	0	1	1	0
597		R3-1->RAM	1	1	0	1	1	0	0	0	0	1	0	0	0
598			1	1	0	1	1	0	0	0	0	1	0	1	0
599		R3-1->MAR	1	1	0	1	1	0	0	0	0	1	1	0	0
600			1	1	0	1	1	0	0	0	0	1	1	1	1
601		PC->RAM	1	1	0	1	1	0	0	1	0	0	0	0	0
602			1	1	0	1	1	0	0	1	0	0	0	1	1
603		IR->PC	1	1	0	1	1	0	0	0	1	0	1	0	0
604			1	1	0	1	1	0	0	1	0	1	1	1	0
605		PC->MAR	1	1	0	1	1	0	0	0	1	1	0	0	0
606			1	1	0	1	1	0	0	0	1	1	0	1	1
607		JP	1	1	0	1	1	0	0	0	1	1	1	0	0

609	RET	SPE2->MAR	1	1	1	0	0	0	0	0	0	0	0	0	0
610			1	1	1	0	0	0	0	0	0	0	0	0	1
611		RAM->R3	1	1	1	0	0	0	0	0	0	0	1	0	0
612			1	1	1	0	0	0	0	0	0	0	1	1	0
613		R3+1->RAM	1	1	1	0	0	0	0	0	0	1	0	0	0
614			1	1	1	0	0	0	0	0	0	1	0	1	0
615		R3->MAR	1	1	1	0	0	0	0	0	0	1	1	0	0
616			1	1	1	0	0	0	0	0	0	1	1	1	0
617		RAM->PC	1	1	1	0	0	0	0	0	1	0	0	0	0
618			1	1	1	0	0	0	0	0	1	0	0	1	0
619		PC->MAR	1	1	1	0	0	0	0	0	1	0	1	0	0
620			1	1	1	0	0	0	0	0	1	0	1	1	0
621		JP	1	1	1	0	0	0	0	0	1	1	0	0	0

形式化的相关表达如下：

(LOOP)

```

11000    ->    SPE->MAR    ->    RAM->R3    ->    R3-1->R3
          ->    SPE->MAR    ->    RAM<-R3    ->    IF0 IR->PC
          ->    PC->MAR    ->    JP
    
```

(PUSH)

```

11001    ->    SPE2->MAR    ->    RAM->R3
          ->    R3-1->RAM    ->    R3-1->MAR    ->    YJP
    
```

(POP)

```

11010    ->    SPE2->MAR    ->    RAM->R3
          ->    R3+1->RAM    ->    R3->MAR    ->    YJP
    
```

(CALL)

```

11011    ->    SPE2->MAR    ->    RAM->R3
          ->    R3-1->RAM    ->    R3-1->MAR
          -> PC->RAM    ->    IR->PC    ->    PC->MAR    ->    JP
    
```

(RET)

```

11010    ->    SPE2->MAR    ->    RAM->R3
          ->    R3+1->RAM    ->    R3->MAR
          ->    PC->MAR    ->    YJP
    
```

最后是 load 和 store 指令的拓展，其中拓展了间接寻址、变址寻址和 PC 相对寻址，关于间接寻址，是将内容从 RAM 中取出后，重新放入到 MAR 中，然后再执行一次访存操作，这时得到的内容才为真正需要的内容。变址寻址是将 R2 作为变址寄存器，与输入的立即数相加，得到真实的访存地址。PC 相对寻址是将 PC 中的值与输入的立即数相加，得到真实的访存地址。由于指令位数的限制，

还可以增加基址寻址，可以让内存中的某一个单元看作基址寄存器，让基址寄存器中的值与立即数相加，得到真实的访存地址。Storeex 指令与 loadex 指令相似，在获取实际地址的过程是完全相同的，不过 laodex 是从内存中读取数据，而 storeex 是将寄存器中的值写入内存中。

623		YJP	1	1	1	0	1	0	0	0	0	0	0	0
624			1	1	1	0	1	0	0	0	0	0	0	0
625		ADDR->MAR	1	1	1	0	1	0	1	0	0	0	0	0
626			1	1	1	0	1	0	1	0	0	0	0	0
627		RAM->MAR	1	1	1	0	1	0	1	0	0	0	0	0
628			1	1	1	0	1	0	1	0	0	0	0	0
629		RAM->R1	1	1	1	0	1	0	1	0	0	0	0	0
630			1	1	1	0	1	0	1	0	0	0	0	0
631		PC->MAR	1	1	1	0	1	0	1	0	0	0	0	0
632			1	1	1	0	1	0	1	0	0	0	0	0
633		JP	1	1	1	0	1	0	1	0	0	0	0	0
634			1	1	1	0	1	0	1	0	0	0	0	0
635		ADDR->R3	1	1	1	0	1	1	0	0	0	0	0	0
636			1	1	1	0	1	1	0	0	0	0	0	1
637		PC+R3->MAR	1	1	1	0	1	1	0	0	0	0	1	0
638	LOADEX		1	1	1	0	1	1	0	0	0	0	1	1
639		RAM->R1	1	1	1	0	1	1	0	0	0	1	0	0
640			1	1	1	0	1	1	0	0	0	1	0	1
641		PC->MAR	1	1	1	0	1	1	0	0	0	1	1	0
642			1	1	1	0	1	1	0	0	0	1	1	1
643		JP	1	1	1	0	1	1	0	0	1	0	0	0
644			1	1	1	0	1	1	0	0	1	0	0	0
645		ADDR->R3	1	1	1	0	1	1	1	0	0	0	0	0
646			1	1	1	0	1	1	1	0	0	0	0	1
647		R2+R3->MAR	1	1	1	0	1	1	1	0	0	0	1	0
648			1	1	1	0	1	1	1	0	0	0	1	1
649		RAM->R1	1	1	1	0	1	1	1	0	0	1	0	0
650			1	1	1	0	1	1	1	0	0	1	0	1
651		PC->MAR	1	1	1	0	1	1	1	0	0	1	1	0
652			1	1	1	0	1	1	1	0	0	1	1	1
653		JP	1	1	1	0	1	1	1	0	1	0	0	0

655		MJP	1	1	1	1	0	0	0	0	0	0	0	0
656														
657		ADDR->MAR	1	1	1	1	0	0	1	0	0	0	0	0
658			1	1	1	1	0	0	1	0	0	0	0	0
659		RAM->MAR	1	1	1	1	0	0	1	0	0	0	0	0
660			1	1	1	1	0	0	1	0	0	0	0	0
661		R1->RAM	1	1	1	1	0	0	1	0	0	0	0	0
662			1	1	1	1	0	0	1	0	0	0	0	0
663		PC->MAR	1	1	1	1	0	0	1	0	0	0	0	0
664			1	1	1	1	0	0	1	0	0	0	0	0
665		JP	1	1	1	1	0	0	1	0	0	0	0	0
666														
667		ADDR->R3	1	1	1	1	0	1	0	0	0	0	0	0
668			1	1	1	1	0	1	0	0	0	0	0	0
669		PC+R3->MAR	1	1	1	1	0	1	0	0	0	0	0	0
670	STOREX		1	1	1	1	0	1	0	0	0	0	0	0
671		R1->RAM	1	1	1	1	0	1	0	0	0	0	0	0
672			1	1	1	1	0	1	0	0	0	0	0	0
673		PC->MAR	1	1	1	1	0	1	0	0	0	0	0	0
674			1	1	1	1	0	1	0	0	0	0	0	0
675		JP	1	1	1	1	0	1	0	0	0	0	0	0
676														
677		ADDR->R3	1	1	1	1	0	1	1	0	0	0	0	0
678			1	1	1	1	0	1	1	0	0	0	0	0
679		R2+R3->MAR	1	1	1	1	0	1	1	0	0	0	0	0
680			1	1	1	1	0	1	1	0	0	0	0	0
681		R1->RAM	1	1	1	1	0	1	1	0	0	0	0	0
682			1	1	1	1	0	1	1	0	0	0	0	0
683		PC->MAR	1	1	1	1	0	1	1	0	0	0	0	0
684			1	1	1	1	0	1	1	0	0	0	0	0
685		JP	1	1	1	1	0	1	1	0	0	0	0	0

由于一共实现了 31 个指令组、80 条指令，由于篇幅的限制不能全部展开解释，完整的指令执行过程详见附录 2：指令执行流程。

3.4 微指令格式

微指令共设计了 24 位，从 0 开始到 23，每一位的解释如下：

第 21-23 位，为 ALU 器件选择，这里采用了一个 3-8 译码器，生成 8 个互斥的信号，基本的原理是 ALU 同时只能执行一种运算，并且只能产生一种结果。其中 0 备用、1 表示选中加法器、2 表示选中移位器、3 表示选中乘法器、4 表示选中除法器、5 表示 JZ 指令、6 表示 JNZ 指令、7 表示需要将全 1 地址送出。

第 15-20 位，为 ALU 控制位，这里的目的是在选中某一个器件后，具体的信号控制，比如当选中加法器后，20-15 位分别表示 M、S3、S2、S1、S0、C0，这些位数恰好与加法器中的控制信号对应，如果选中移位器，则 20-17 位分别表示 L、R、S0、S1。如果选中其他器件，则改部分全部设置位 0。

第 13-14、11-12 位为 A 选择器选择和 B 选择器选择，均采用 2-4 译码器解码，对于 A 选择器来说，0 备用、1 选中 RAM、2 选中 R1、3 选中 R3，对于 B 选择器来说，0 备用、1 选中 PC、2 选中 R2、3 选中 IR。

第 8-10 位为输出分配，这里采用 3-8 译码器进行解码，同样 0 表示备用、1 表示输出到 RAM 中、2 表示输出到 R1 中、3 表示输出到 R2 中、4 表示输出到 R3

中、5 表示输出到 PC 中、6 表示输出到 IR（没用）、7 表示输出到 MAR 中。

第 6-7 位为读写信号，7 位 1 表示写、0 表示读。

第 5 位为停机指令，1 表示停机。

第 2-4 位为后继地址生成部件，采用 3-8 译码器进行解码，0 备用、1 表示采用 upc+1 方式、2 表示 JP 无条件跳转（这里只能是跳转到全 0，取指操作）、3 表示 QJP，根据 OP 跳转到程序入口处、4 表示 YJP 根据源操作数进一步跳转、5 表示 MJP 根据目的操作数进行进一步跳转、6 表示 MOVJP 专门为 MOV 指令设计的微程序的跳转。

第 1 位是专门为 MOV 指令设置的判断位，这一位的目的是，在通常情况下所有的立即数或者地址均是 8 位的，但是在 MOV 指令中，由于需要进行寄存器的选择，因此会占用其中的两位，这两位不能直接送到 B 选择器中，因此需要置 0，特设置了这一位用来将 IR 的某两位置 0。

第 0 位是将 sp 的地址（即栈顶寄存器）送出 ALU 的信号，在 push、pop、call、ret 指令中都体现。

S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP
微指令																							
器件选择		ALU控制位						A选择		B选择		输出分配		WREN		停机		后继地址		MO		SPE	
21-23		15-20						13-14		11-12		8-10		6-7		5		2-4		1		0	
3-8译码		M	S3	S2	S1	S0	C0	2-4译码	2-4译码	3-8译码								3-8译码					
		L	R	S0	S1	/	/																
加法器 001		加法器M S3 S2 S1 S0 C0 移位器 L R S0 S1 其他均为0						RAM 01 R1 10 R3 11		PC 01 R2 10 IR 11		RAM 001 R1 010 R2 011 R3 100 PC 101 IR 110 MAR 111		/		W	1停	+1 001 JP 010 QJP 011 YJP 100 MJP 101 MOVJP 110		3.4 是否有效		SPE 2	
移位器 010																							
乘法器 011																							
除法器 100																							
JZ 101																							
JNZ 110																							
CX 111																							

上图是微指令的详细介绍，下面给出部分指令执行时的微指令作为展示，全部的微指令见附录 3：微程序。

（1）取指

#	A	B	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP					
1	指令	微操作	微指令B	器件选择		ALU控制位						A选择		微指令B选择		输出分配		WREN		停机	后继地址		MO	SPE								
2				21-23		15-20						13-14		11-12		8-10		6-7		5	2-4		1	0								
3				3-8译码		M	S3	S2	S1	S0	C0	2-4译码		2-4译码		3-8译码						3-8译码										
4						L	R	S0	S1	/	/																					
5				加法器 001 移位器 010 乘法器 011 除法器 100 JZ 101 JNZ 110 CX 111		加法器M S3 S2 S1 S0 C0 移位器 L R S0 S1 其他均为0						RAM 01 R1 10 R3 11		PC 01 R2 10 IR 11		RAM 001 R1 010 R2 011 R3 100 PC 101 IR 110 MAR 111		/		W	1停	+1 001 JP 010 QJP 011 YJP 100 MJP 101 MOVJP 110		3.4 是否有效	SPE 2							
6																																
7				取指	RAM->IR	001111110010000000000100	0	0	1	1	1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0		
8						0011111100100110000000100	0	0	0	1	1	1	1	1	1	0	0	1	0	0	1	1	0	0	0	0	0	0	1	0	0	
9						0010100100001000000000100	0	0	0	1	0	1	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0
10						0+PC+1->PC	0010100100001101000000100	0	0	0	1	0	1	0	0	1	0	0	0	0	1	1	0	1	0	0	0	0	0	1	0	0
11	QJP	0000000000000000000001100	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0		

（2）加法（减法、逻辑运算、自增、自减等其他利用加法器实现的指令，暂不展示，详细见附件 3）

[illegible][illegible][illegible]

[illegible][illegible]

455	STORE	IR->MAR	001110100001100000000100	0	0	0	1	1	1	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0		
456				00111010000111110000000100	0	0	0	1	1	1	0	1	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	1	0	0	
457			MJP	0000000000000000000010100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	
458																																
459				001111110100000001000100	0	0	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0
460				001111110100000101000100	0	0	0	1	1	1	1	1	1	0	1	0	0	0	0	0	1	0	1	0	1	0	0	0	1	0	0	0
461				0011101000001000000000100	0	0	0	1	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0
462			PC->MAR	001110100000111100000100	0	0	0	1	1	1	0	1	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	1	0	0	0
463																																
464			JP	0000000000000000000001000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
465																																
466			R2->RAM	001110100001000001000100	0	0	0	1	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0
467				001110100001000101000100	0	0	0	1	1	1	0	1	0	0	0	0	1	0	0	0	1	0	1	0	0	0	1	0	0	1	0	0
468			PC->MAR	0011101000001000000000100	0	0	0	1	1	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0
469				001110100000111100000100	0	0	0	1	1	1	0	1	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	1	0	0	0
470			JP	0000000000000000000001000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
471																																
472		R3->RAM	001111110110000001000100	0	0	0	1	1	1	1	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	0
473			001111110110000101000100	0	0	0	1	1	1	1	1	1	0	1	1	0	0	0	0	1	0	1	0	0	0	1	0	0	1	0	0	
474		PC->MAR	0011101000001000000000100	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	
475																																

[illegible]

[illegible][illegible][illegible]

32

ROM 的部分设置如下，由于微程序设计原因，其中内容并不连续，因此只展示部分结果，完整的 ROM 详见项目工程文件。

34

3.5 RAM 应用程序设计

3.5.1 递推计算斐波那契数列

下面是计算斐波那契数列第 n 项的汇编语言代码。

Calculation of the Fibonacci sequence

INPUT: N : The n th term of the Fibonacci sequence

OUTPUT: $R2$: The value of the n th term of the Fibonacci sequence

```

1:  MOV    R1,N-2    ;set the number of N
2:  STORE  CX,R1     ;move N to the CX
3:  MOV    R1,1      ;set the origin number of r1
4:  MOV    R2,1      ;set the origin number of r2
5:  ADD    R1,R2     ;r1+r2->r1, ans in r1
6:  XCHG                   ;move the ans to r2, the old one to r1
7:  LOOP   0X04H     ;add again
8:  HLT
    
```

机器语言如下：

地址H	地址B	指令	代码B	代码H
0x0	000	MOV R1,N-2	1011100101000110	0xB946
0x1	001	STORE R1,CX	1011000111111111	0xB1FF
0x2	010	MOV R1,1	1011100101000001	0xB941
0x3	011	MOV R2,1	1011100110000001	0xB981
0x4	100	ADD R1,R2	0000101000000000	0xA0
0x5	101	XCHG	1010000000000000	0xA00
0x6	110	LOOP 100	1100000000000100	0xC04

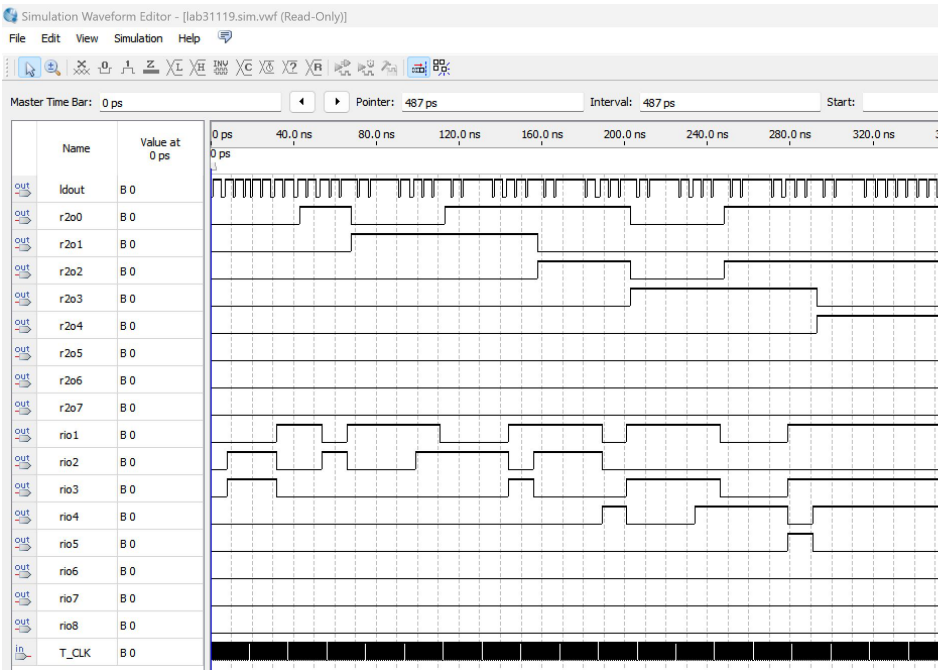
RAM 设置如下：

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
00000	B946	B1FF	B941	B981	0A00	A000	C004	0000	

上面的代码巧妙的利用了 XCHG 指令的作用，和 ADD 指令的特性，由于 ADD 指令每一次都会将 R1 中的值覆盖，而斐波那契的计算过程是 $f(n)=f(n-2)+f(n-1)$ ，所以只需要保证我们将 $f(n-2)$ 替换为 $f(n)$ 即可，但是在每一次计算结束后，R1 寄存器中的值都会变为新的值，即下一轮的 $f(n-1)$ ，而 R2 寄存器中的值会变位下一轮的 $f(n-2)$ ，所以这时就要用 XCHG 指令，将 R1 与 R2 寄存器中的值进行互换，从而保持循环不变量的特性。

结果：按照递推的方式计算斐波那契的第 8 项。R2 中先后出现 1、1、2、3、5、8、13、21。

模拟仿真结果如下（板子上实验结果与模拟仿真相同，已验收）：



3.5.2 递推计算阶乘

下面是递推计算 n 的阶乘的汇编语言代码。

Calculation of the factorial of n

INPUT: N : The number wanted to calculate the factorial

OUTPUT: $R1$: The value of the factorial of n

```
1:  MOV    R1,N      ;set the number of N
2:  STORE  CX,R1     ;move N to the CX
3:  MOV    R1,1      ;set the origin number of r1
4:  MOV    R2,1      ;set the origin number of r2
5:  MUL    R1,R2     ;r1*r2->r1, ans in r1
6:  INC    R2        ;R2++
7:  LOOP   0X04H     ;multiply again
8:  HLT
```

机器语言如下:

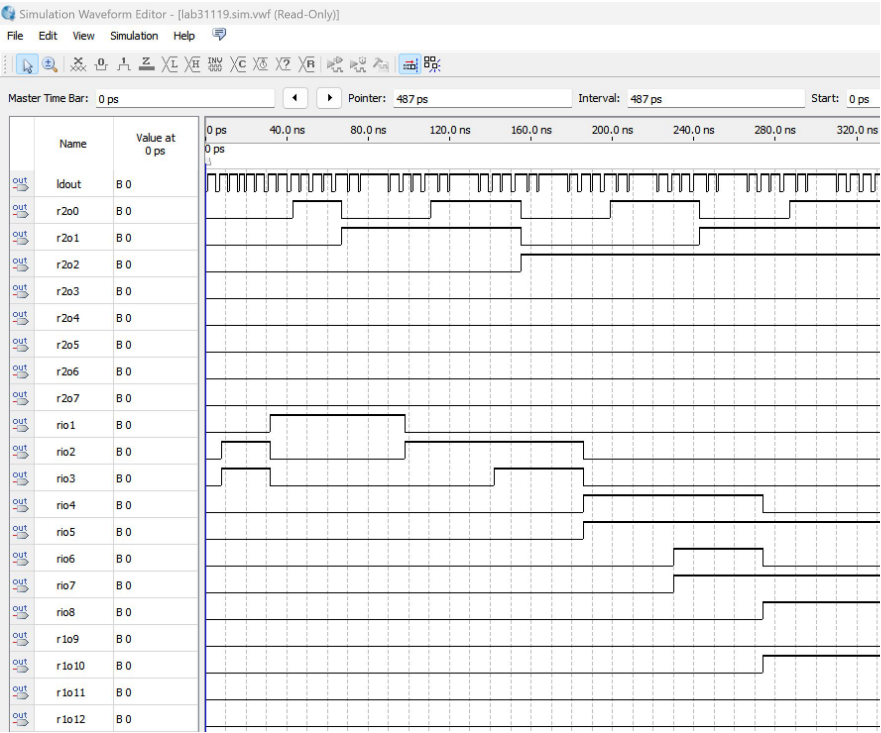
26	0x0	000	MOV R1,N	1011100101000110	0xB946
27	0x1	001	STORE R1,CX	1011000111111111	0xB1FF
28	0x2	010	MOV R1,1	1011100101000001	0xB941
29	0x3	011	MOV R2,1	1011100110000001	0xB981
30	0x4	100	MUL R1,R2	0001101000000000	0x1A00
31	0x5	101	INC R2	0010101000000000	0x2A00
32	0x6	110	LOOP 100	1100000000000100	0xC004
33	0x7	111	HLT	1111100000000000	0xF800

在上面的过程中计算 6 的阶乘，结果保存在 $R1$ 中，每一轮让 $R2$ 自增 1，实现 $1*2*3*4*5*6$ 的操作。最终得到计算结果为 720，转换为二进制为 0010 1101 0000。

RAM 内容如下:

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
00000	B946	B1FF	B941	B981	1A00	2A00	C004	F800	

模拟仿真结果如下（板上实验结果与模拟仿真相同，已验收）：



3.5.3 其他指令的混合测试

下面是一些混合指令的测试没有实际意义，但是可以验证指令的正确性。

Complex testing of instructions

- 1: MOV R1,4
- 2: STORE CX,R1
- 3: MOV R2,0X0DH
- 4: LOAD R1,CX
- 5: DIV R1,2
- 6: CMP R1,1
- 7: JZ 1101
- 8: SHL R2
- 9: JMP 1011
- 10: SHL R2
- 11: SHL R2
- 12: LOOP 0X03H
- 13: HLT

机器语言如下：

51	0x0	0000	MOV R1,4	1011100101000100	0xB944
52	0x1	0001	STORE R1,CX	1011000111111111	0xB1FF
53	0x2	0010	MOV R2,1101	1011100110001101	0xB98D
54	0x3	0011	LOAD R1,CX	1010100111111111	0xA9FF
55	0x4	0100	DIV R1,2	0010000100000010	0x2102
56	0x5	0101	CMP R1,1	0100000100000001	0x4101
57	0x6	0110	JZ 1001	1001000000001001	0x9009
58	0x7	0111	SHL R2	0110101000000000	0x6A00
59	0x8	1000	JMP 1011	1000100000001011	0x880B
60	0x9	1001	SHR R2	0111101000000000	0x7A00
61	0xA	1010	SHR R2	0111101000000000	0x7A00
62	0xB	1011	LOOP 0011	1100000000000011	0xC003
63	0xC	1100	HTL	1111100000000000	0xF800

RAM 中结果如下：

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
0	B944	B1FF	B98D	A9FF	2102	4101	9009	6A00	
8	880B	7A00	7A00	C003	F800	0000	0000	0000	

该段代码设置 CX 计数器的值为 4，即重复执行 4 次，R2 寄存器设置初值为 1101，在 while 循环中，每次获取循环变量的值放入到 R1 中，并且让 R1/2，如果计算的结果等于 1，则右移两次，否则左移一次。

第一次 CX=4， $4/2=1$ ，R2 左移一次，1101→11010

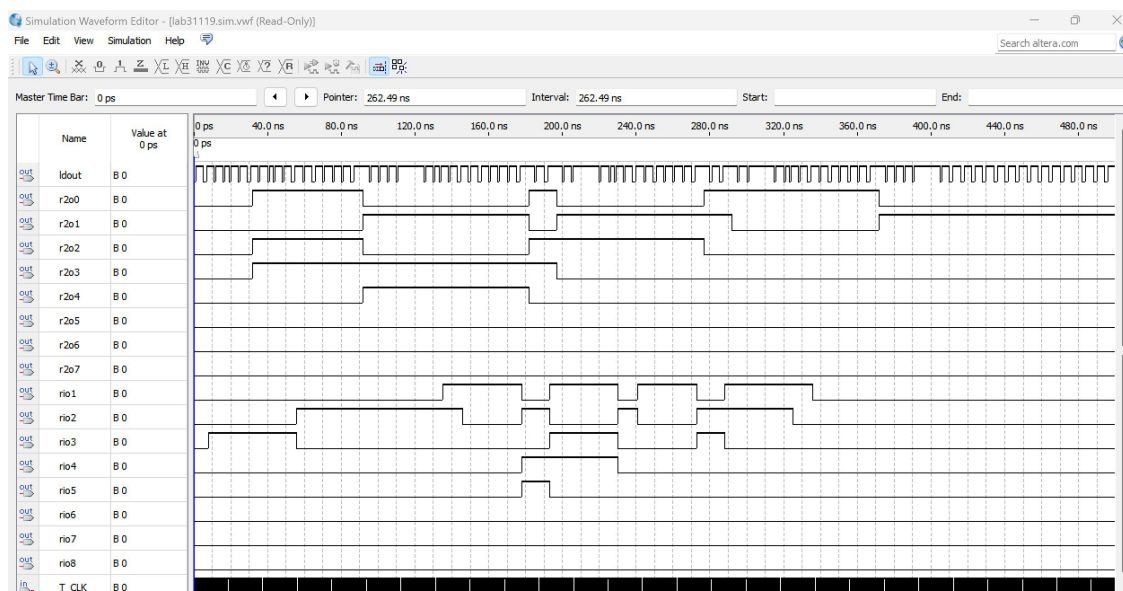
第二次 CX=3， $3/2=1$ ，R2 右移两次，11010→0110

第三次 CX=2， $2/2=1$ ，R2 右移两次，0110→0001

第四次 CX=1， $1/2=0$ ，R1 左移一次，0001→0010

退出循环，程序终止。

模拟仿真结果如下（板上实验结果与模拟仿真相同，已验收）：



3.5.4 基于全局变量传参的子程序递归计算阶乘

下面是利用全局变量传参的子程序递归计算 n 的阶乘的汇编语言代码。

Calculation of the factorial of n

INPUT: N : The number wanted to calculate the factorial

OUTPUT: $R1$: The value of the factorial of n

```

1:  ADD    R1,11111111
2:  STORE  R1,01111111 ;set the origin number of stack for call
3:  MOV    R2,N
4:  MOV    R1,1
5:  STORE  R1,00100000
6:  STORE  R2,00100001
7:  CALL   1000
8:  HLT

9:  LOAD   R1,00100001
10: CMP    R1,1
11: JZ     10010
12: MOV    R2,R1
13: LOAD   R1,00100000
14: MUL    R1,R2
15: STORE  R1,00100000
16: DEC    R2
17: STORE  R2,00100001
18: CALL   1000
19: RET
    
```

机器语言如下：

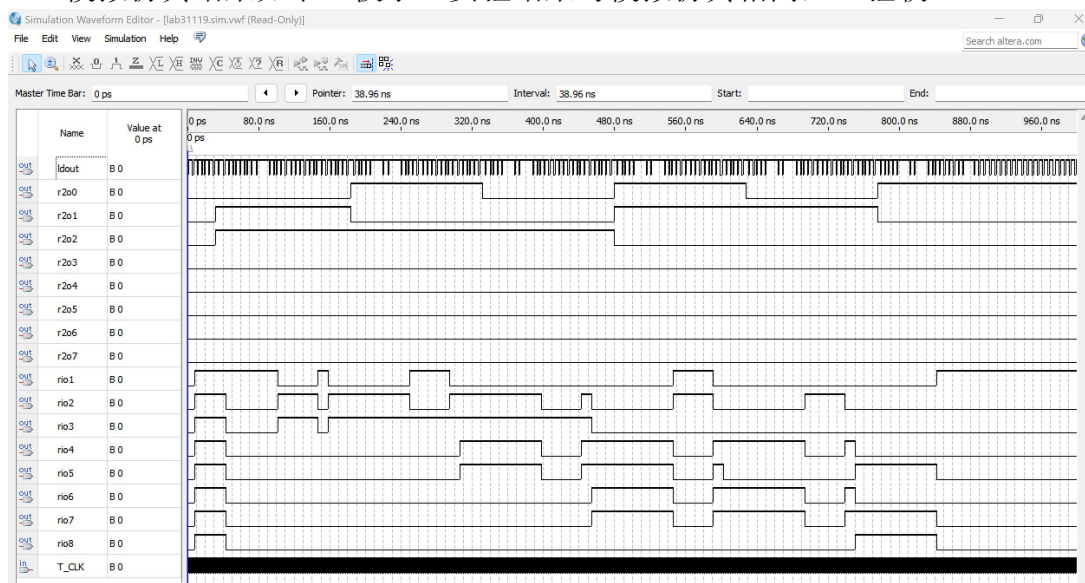
98	0x00	00000000	ADD R1,11111111	0000100111111111	0x09FF
99	0x01	00000001	STORE R1,01111111	1011000101111111	0xB17F
100	0x02	00000010	MOV R2,N	1011100110000101	0xB985
101	0x03	00000011	MOV R1,1	1011100101000001	0xB941
102	0x04	00000100	STORE R1,00100000	1011000100100000	0xB120
103	0x05	00000101	STORE R2,00100001	1011001000100001	0xB221
104	0x06	00000110	CALL 1000	1101100000001000	0xD808
105	0x07	00000111	HLT	1111100000000000	0xF800
106	0x08	00001000	LOAD R1,00100001	1010100100100001	0xA921
107	0x09	00001001	CMP R1,1	0100000100000001	0x4101
108	0x0A	00001010	JZ 10010	1001000000010010	0x9012
109	0x0B	00001011	MOV R2,R1	1011101010000100	0xBA84
110	0x0C	00001100	LOAD R1,00100000	1010100100100000	0xA920
111	0x0D	00001101	MUL R1,R2	0001101000000000	0x1A00
112	0x0E	00001110	STORE R1,00100000	1011000100100000	0xB120
113	0x0F	00001111	DEC R2	0011001000000000	0x3200
114	0x10	00010000	STORE R2,00100001	1011001000100001	0xB221
115	0x11	00010001	CALL 1000	1101100000001000	0xD808
116	0x12	00010010	RET	1110000000000000	0xE000

该段代码中将阶乘的计算结果放到了 00100000 地址中，而传递的参数（当前需要乘的数）放到了 00100001 地址中，在 call 指令中先判断当前乘的数字是否为 1，如果是 1 则直接跳转到 RET 指令出返回，否则从内存中取出上一轮阶乘的结果，并且与当前位数进行相乘，保存结果，并且让下一次乘的数-1。

RAM 中结果如下：

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
00000	09FF	B17F	B985	B941	B120	B221	D808	F800	
00008	A921	4101	9012	BA84	A920	1A00	B120	3200	

模拟仿真结果如下（板上实验结果与模拟仿真相同，已验收）：



上面仍然计算 6 的阶乘，不过只展示了低 8 位，结果是正确的。

3.5.5 基于栈传参的子程序递归计算阶乘

下面是利用栈传参的子程序递归计算 n 的阶乘的汇编语言代码。

Calculation of the factorial of n

INPUT: N : The number wanted to calculate the factorial

OUTPUT: $R1$: The value of the factorial of n

```

1:  ADD    R1,11111111
2:  STORE  R1,01111111 ;set the origin number of stack for call
3:  MOV    R2,N
4:  MOV    R1,1
5:  PUSH   R1
6:  PUSH   R2
7:  CALL   1010
8:  POP    R2
9:  POP    R1
10: HLT

11: LOAD   R2,SP
12: LOADEX R1,[R2+2]
13: STORE  R1,00100000
14: LOADEX R1,[R2+1]
15: CMP    R1,1
    
```

```

16: JNZ      10001
17: RET
18: MOV      R2,R1
19: LOAD      R1,00100000
20: MUL      R1,R2
21: PUSH      R1
22: DEC      R2
23: PUSH      R2
24: CALL      1010
25: POP      R2
26: POP      R1
27: LOAD      R2,SP
28: STOREEX   R1,[R2,+2]
29: RET
    
```

机器语言如下：

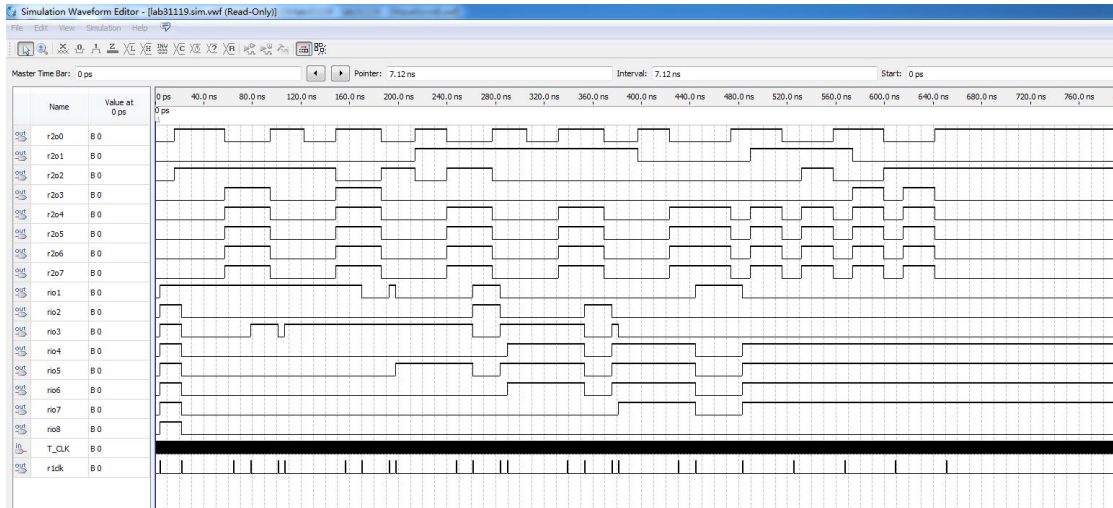
65	0x00	00000000	ADD R1,11111111	0000100111111111	0x09FF
66	0x01	00000001	STORE R1,01111111	1011000101111111	0xB17F
67	0x02	00000010	MOV R2,N	1011100110000101	0xB985
68	0x03	00000011	MOV R1,1	1011100101000001	0xB941
69	0x04	00000100	PUSH R1	1100100100000000	0xC900
70	0x05	00000101	PUSH R2	1100101000000000	0xCA00
71	0x06	00000110	CALL 1010	1101100000001010	0xD80A
72	0x07	00000111	POP R2	1101001000000000	0xD200
73	0x08	00001000	POP R1	1101000100000000	0xD100
74	0x09	00001001	HLT	1111100000000000	0xF800
75	0x0A	00001010	LOAD R2,SP	1010101001111111	0xAA7F
76	0x0B	00001011	LOAD R1,[R2+2] (变址寻址, 取R1)	1110101100000010	0xEB02
77	0x0C	00001100	STORE R1,00100000	1011000100100000	0xB120
78	0x0D	00001101	LOAD R1,[R2+1] (变址寻址, 取R2)	1110101100000001	0xEB01
79	0x0E	00001110	CMP R1,1	0100000100000001	0x4101
80	0x0F	00001111	JNZ 10001	1001100000010001	0x9811
81	0x10	00010000	RET	1110000000000000	0xE000
82	0x11	00010001	MOV R2,R1	1011101010000100	0xBA84
83	0x12	00010010	LOAD R1,00100000	1010100100100000	0xA920
84	0x13	00010011	MUL R1,R2	0001101000000000	0x1A00
85	0x14	00010100	PUSH R1	1100100100000000	0xC900
86	0x15	00010101	DEC R2	0011001000000000	0x3200
87	0x16	00010110	PUSH R2	1100101000000000	0xCA00
88	0x17	00010111	CALL 1010	1101100000001010	0xD80A
89	0x18	00011000	POP R2	1101001000000000	0xD200
90	0x19	00011001	POP R1	1101000100000000	0xD100
91	0x1A	00011010	LOAD R2,SP	1010101001111111	0xAA7F
92	0x1B	00011011	STORE R1,[R2+2] (变址寻址, 改R1)	1111001100000010	0xF302
93	0x1C	00011100	RET	1110000000000000	0xE000

该段代码中将阶乘的计算结果放到 R1 寄存器中，乘数放到 R2 寄存器中，每次调用该函数时，都会先将 R1 和 R2 寄存器进行压栈，然后通过变址寻址访存到传递的参数。然后正常计算结果，重复上面的过程，直到乘数变成 1 则子程序返回。当子程序返回时，需要修改 R1 对应的地址，这一步相当于 return 需要的结果。重复上面的过程，最后 R1 寄存器中将会得到计算的结果。

RAM 中结果如下：

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
00000	B986	B941	C900	CA00	D808	D100	D200	F800	
00008	AA7F	EB02	B120	EB01	4101	980F	E000	BA84	
00010	A920	1A00	C900	3200	CA00	D808	D200	D100	
00018	AA7F	F302	E000	0000	0000	0000	0000	0000	

模拟仿真结果如下（板上实验结果与模拟仿真相同，已验收）：



由于 FPGA 板上的实验结果难以拍照记录，这里不进行展示，但是可以保证的是，上面所有的实验结果均已验收，且 FPGA 板上的结果与模拟仿真的结果一致。

第 4 章 基于硬布线实现的模型机指令与结构的修改

在微程序的实现过程中,利用指令的操作码和微地址形成部件实现了分支跳转的功能,即可以根据操作数的类型的不同进而跳转到相同的程序入口地址的不同分支中,从而实现了指令组的这一特性,即操作码相同,但是可以执行不同的寻址方式,比如 MOV 指令组,只有一个 OP,但是可以执行直接寻址、寄存器寻址等,LOADEX/STOREEX 指令组,只有一个 OP,但是可以执行 PC 相对寻址、间接寻址、变址寻址等。

但是由于硬布线代替了原有的微程序结构,因此分支跳转难以实现,所以对于前面的指令格式需要进行部分的修改,以适应基于硬布线实现的模型机。

4.1 指令格式修改

为了方便起见,我们将指令字长设置为了 8 位,但是其中设置了单字长指令和双字长指令。所有的指令只有高四位为操作码,其他位数均为操作数,根据不同的数量的地址设置了不同类型的指令。

对于 ADD、SUB、MUL、DIV 等运算指令来说,均为单字长指令。其中单字长指令根据操作数又可以分为零地址指令,其高 4 位为操作码,其他位数备用,例如 HLT 指令。对于一地址指令,高 4 位为操作码,低 4 位指示其操作数,例如 DEC 指令。对于二地址指令,高 4 位为操作码,中间 2 位为操作数 1,低 2 位为操作数 2,如 ADD 指令。对于双字长指令来说,高 4 位为操作码、中间 4 位为操作数 1、最低 8 位为操作数 2,例如 MOV 指令。

7	6	5	4	3	2	1	0
操作码OP				/			
7	6	5	4	3	2	1	0
操作码OP				操作数			
7	6	5	4	3	2	1	0
操作码OP				数1		数2	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
操作码OP				操作数1				操作数2							

4.2 指令介绍

由于操作码设置为了 4 位,一共可以实现 16 条指令,其中包括了 12 条算数和逻辑运算指令、3 条取数指令和 1 条停机指令。

其中所有的双操作数运算,如 ADD、SUB、MUL、DIV、AND、OR、XOR、NOR 指令均是 R0 寄存器与 R1 寄存器执行运算,并且将结果送到 R0 寄存器中。

对于单操作数的运算,如 INC、DEC、SHL、SHR 指令均是 R0 寄存器中的数据执行某种运算后,结果保存在 R0 中。

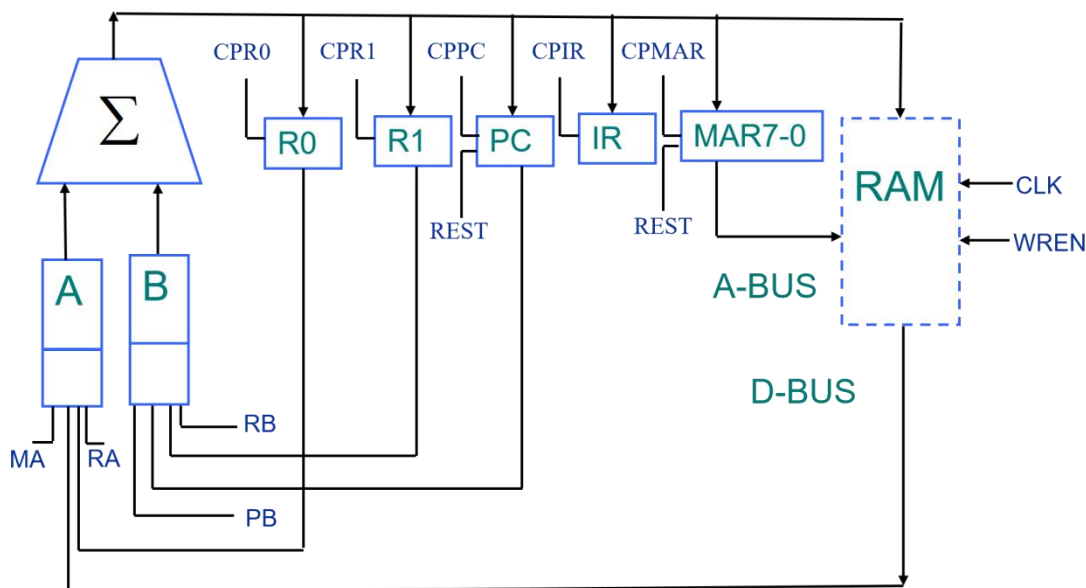
最特殊的是 MOV 指令,对于 MOV1 操作来说,将一个立即数送入到 R0 中;对

于 MOV2 指令来说，将立即数送入到 R1 中；对于 MOV3 指令来说，将 R0 中的值，存放到给定的地址中，这个地址是 8 位的。

ADD	加法指令		MOV1	送数指令1
SUB	减法指令		MOV2	送数指令2
MUL	乘法指令		MOV3	送数指令3
DIV	除法指令			
INC	自增指令		HLT	停机指令
DEC	自减指令			
AND	求与指令			
OR	求或指令			
XOR	异或指令			
NOR	或非指令			
SHL	逻辑左移指令			
SHR	逻辑右移指令			

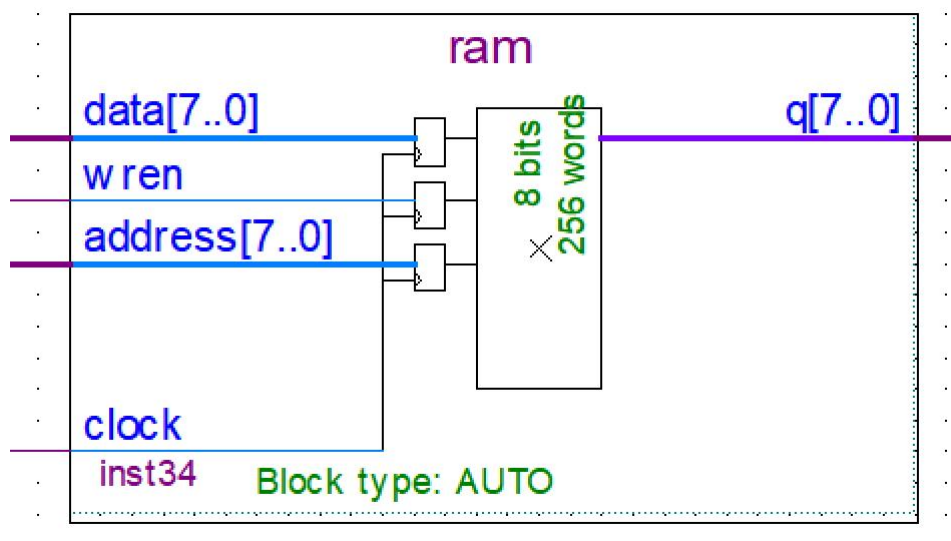
4.3 总体结构修改

由于硬布线实现的指令较为简单，这里只设置了两个通用寄存器 R0 和 R1，因此没有额外的 R3 寄存器，并且所有的立即数均是通过双字长指令给出，因此也不存在 IR→ALU 的数据通路，所以总体通路如下：



4.4 RAM 修改

由于这里的指令长度修改为了 8 位，并且指令能够访存的大小也变成了 $2^8=256$ words，所以 RAM 大小实际为 256 words*8 bits，RAM 器件如下：



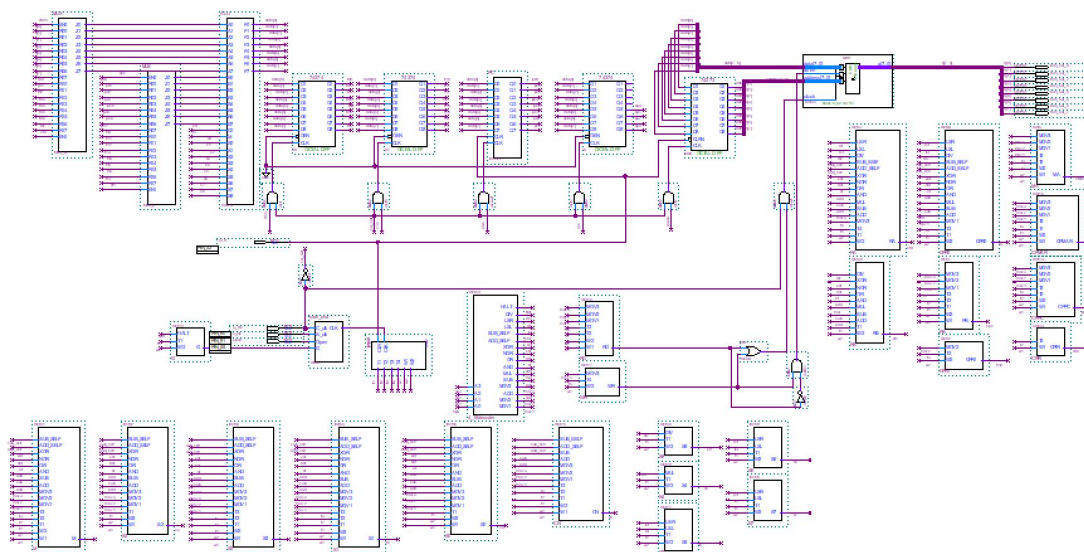
4.5 其他说明

在数据通路上，因为其影响较小，所以不再额外绘制，只不过能够送数的寄存器只有 R0、R1、PC，能够接收数据的寄存器只有 R0、R1、PC、IR、MAR。

除此之外，由于其他组件完全相同，如译码器、ALU、寄存器、数据选择器等，所以这里也不额外展示其组成结构详细内容见 2.3

4.6 总体原理图

基于前面所述的总体结构、数据通路和器件变化，以及指令长度和形式的变化，修改后的总体原理图如下，其中右下角的微程序控制器被替换成了硬布线实现所产生的逻辑信号，并且与对应的控制器件相连。

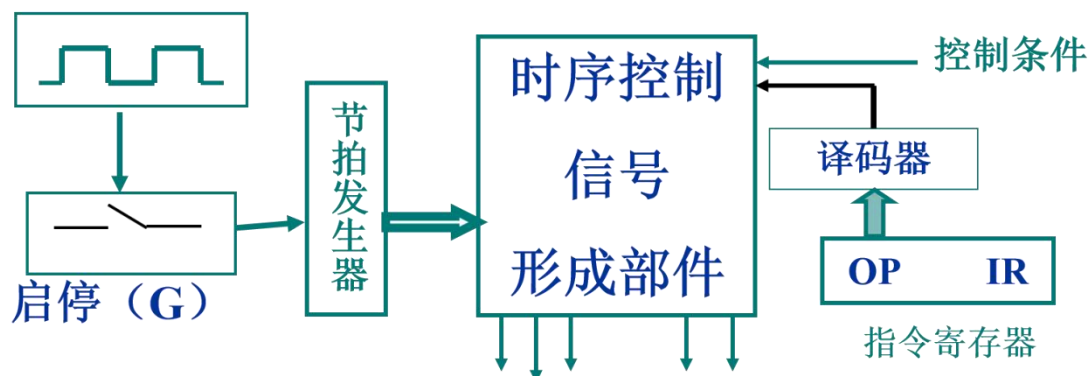


第 5 章 硬布线实现的控制部件

5.1 硬布线实现的 CU 框图

硬布线实现的 CU 框图如下：

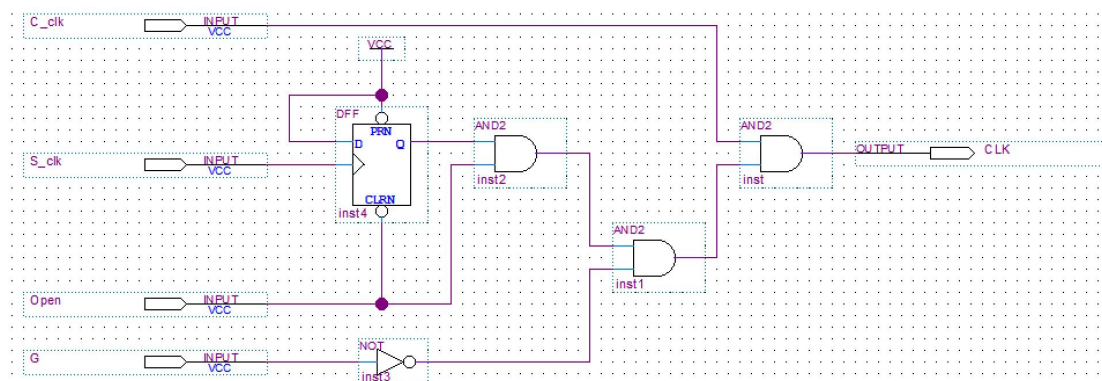
主振（连续脉冲）



5.2 硬布线控制器器件介绍

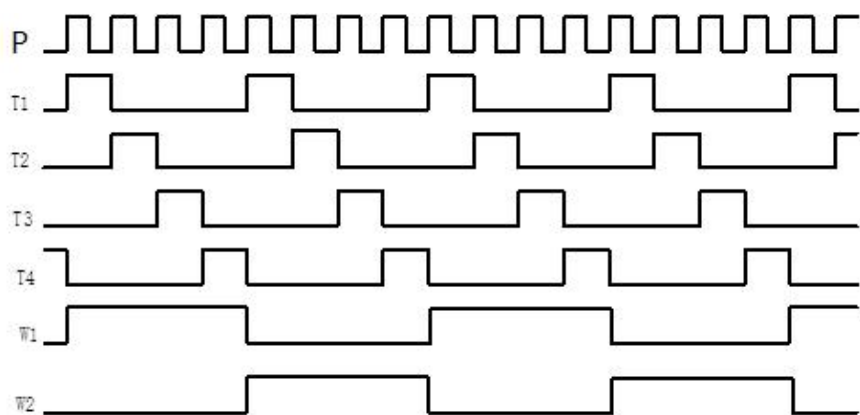
5.2.1 启动电路

为了能够使用连续的脉冲，设置了启动电路，使之能够产生系统电路所需要的连续脉冲，原理图如下：

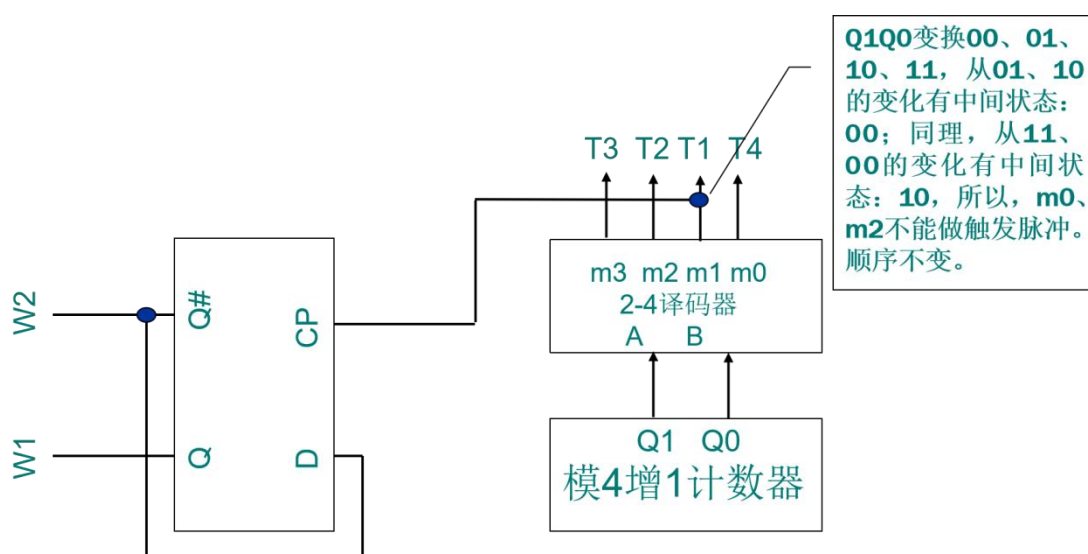


5.2.2 节拍发生器

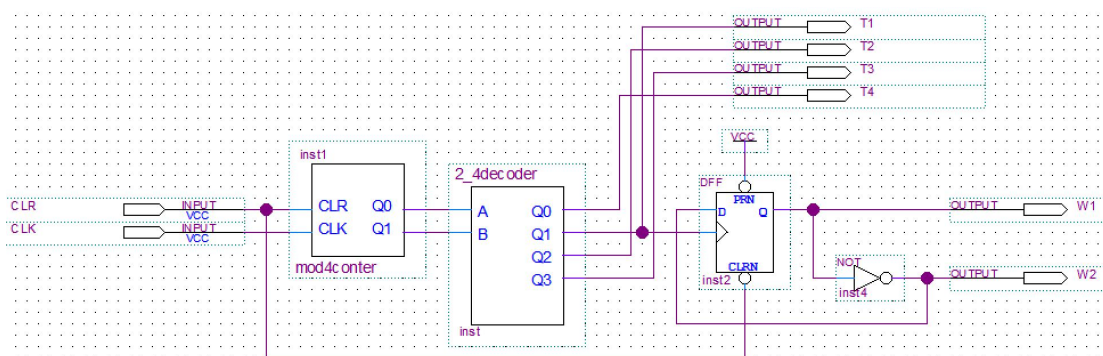
根据指令执行流程，设计的时序分为取指周期和执行周期，每个周期均为 4 个节拍，设置的波形图如下：



根据设计的波形图，进而设置的节拍发生器的框图如下：



根据设计的框图，进而设置的节拍发生器的原理图如下：



5.2.3 指令译码器

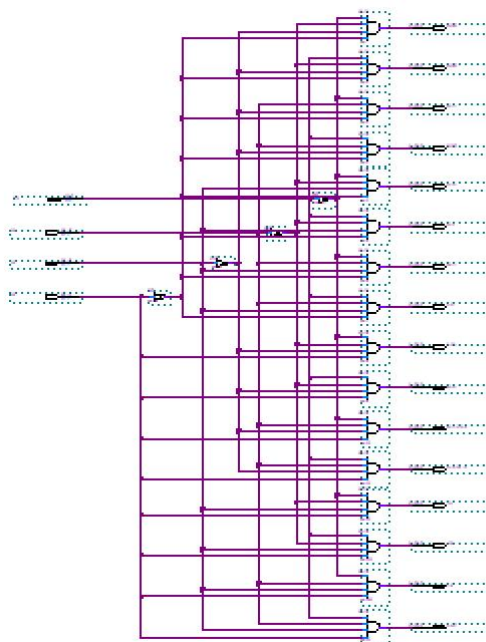
利用指令的操作码，得出逻辑表达式如下：

$$OP = 0000 \quad MOV1 = \overline{IR_7} \overline{IR_6} \overline{IR_5} \overline{IR_4}$$

$$OP = 0001 \quad MOV2 = \overline{IR_7} \overline{IR_6} \overline{IR_5} IR_4$$

OP = 0010	ADD = $\overline{IR_7} \overline{IR_6} IR_5 \overline{IR_4}$
OP = 0011	MOV3 = $\overline{IR_7} \overline{IR_6} IR_5 IR_4$
OP = 0100	SUB = $\overline{IR_7} IR_6 \overline{IR_5} \overline{IR_4}$
OP = 0101	MUL = $\overline{IR_7} IR_6 \overline{IR_5} IR_4$
OP = 0110	AND = $\overline{IR_7} IR_6 IR_5 \overline{IR_4}$
OP = 0111	OR = $\overline{IR_7} IR_6 IR_5 IR_4$
OP = 1000	NOR = $IR_7 \overline{IR_6} \overline{IR_5} \overline{IR_4}$
OP = 1001	XOR = $IR_7 \overline{IR_6} \overline{IR_5} IR_4$
OP = 1010	INC = $IR_7 \overline{IR_6} IR_5 \overline{IR_4}$
OP = 1011	DEC = $IR_7 \overline{IR_6} IR_5 IR_4$
OP = 1100	SHL = $IR_7 IR_6 \overline{IR_5} \overline{IR_4}$
OP = 1101	SHR = $IR_7 IR_6 \overline{IR_5} IR_4$
OP = 1110	DIV = $IR_7 IR_6 IR_5 \overline{IR_4}$
OP = 1111	HLT = $IR_7 IR_6 IR_5 IR_4$

可以看到其实操作码的逻辑表达式所组成的与、非门设计的电路，恰好与4-16译码器的功能相同，所以电路图如下：



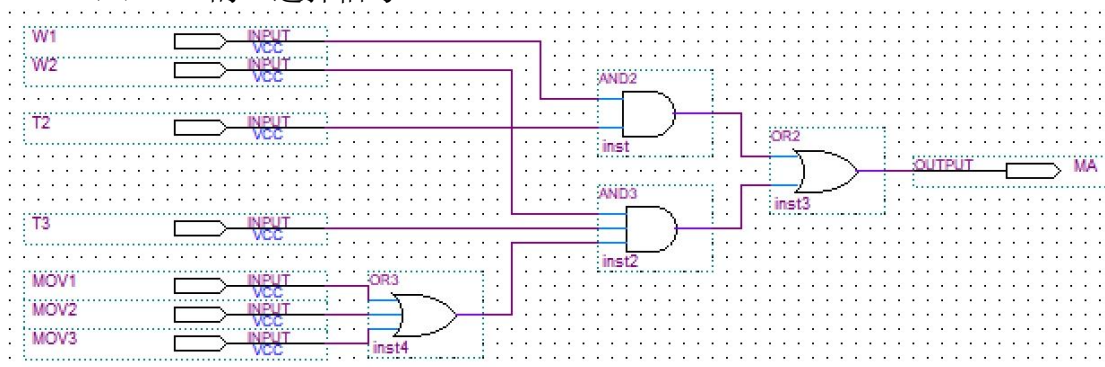
5.2.4 时序控制信号形成部件

时序控制信号信号形成部件，由多个与非门组合而成，是先根据指令执行的流程和数据通路，再加上每一个微操作的控制信号形成的控制信号列表，进而给

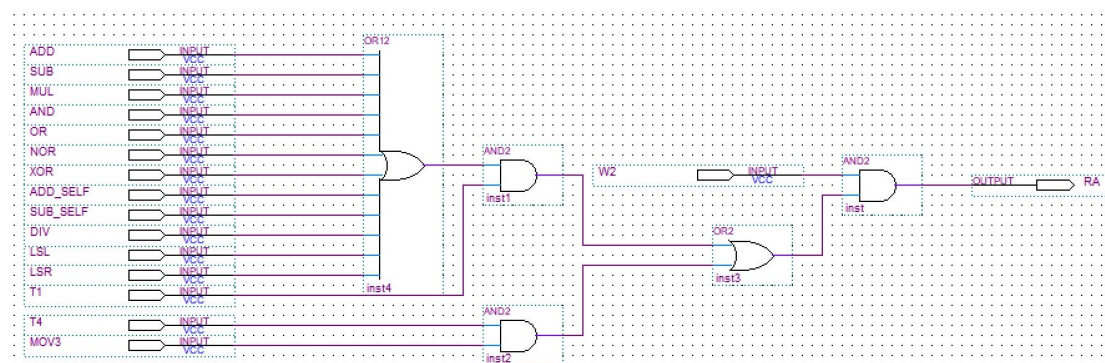
出各个控制信号的逻辑表达式，最后根据逻辑表达式连接出的原理图。具体的设计详见 5.3 和 5.4，这里仅给出其原理图。

(一) 选择信号：

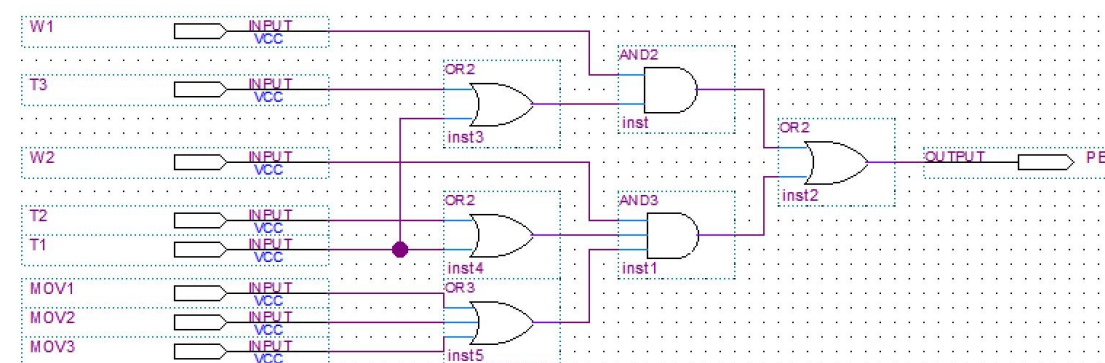
(1) RAM 的 A 选择信号 MA



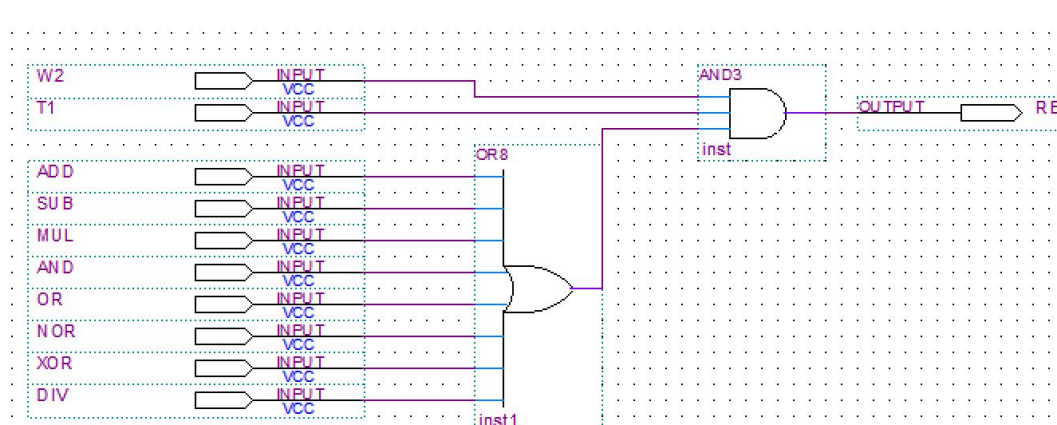
(2) R0 的 A 选择信号 RA



(3) PC 的 B 选择信号 PB



(4) R1 的 B 选择信号 RB



(1) R0 的激活信号 CPR0



W2



.....

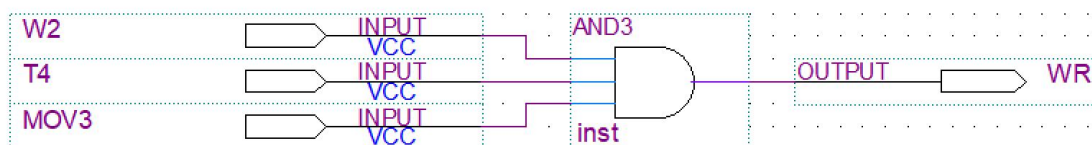


.....

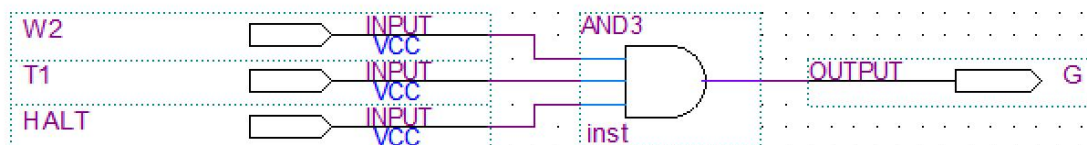


.....

(7) RAM 写信号 WR

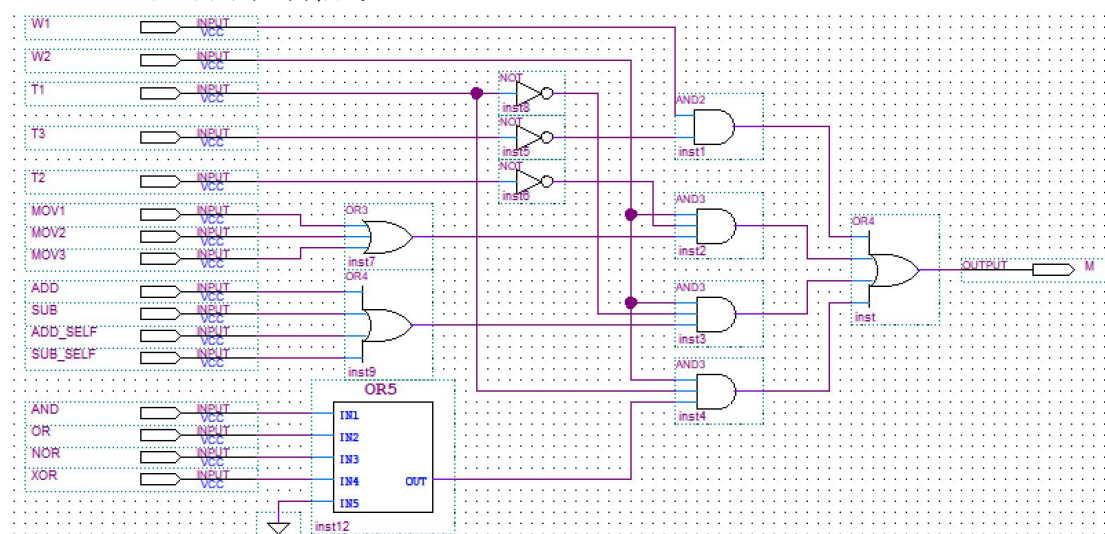


(三) 停机信号:

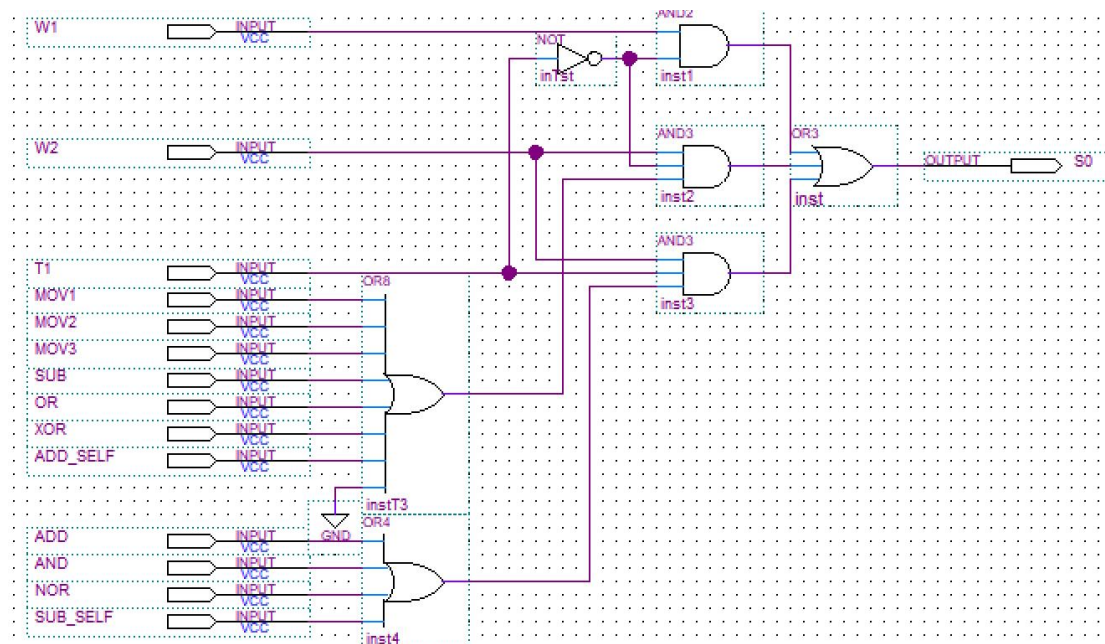


(四) 运算控制信号

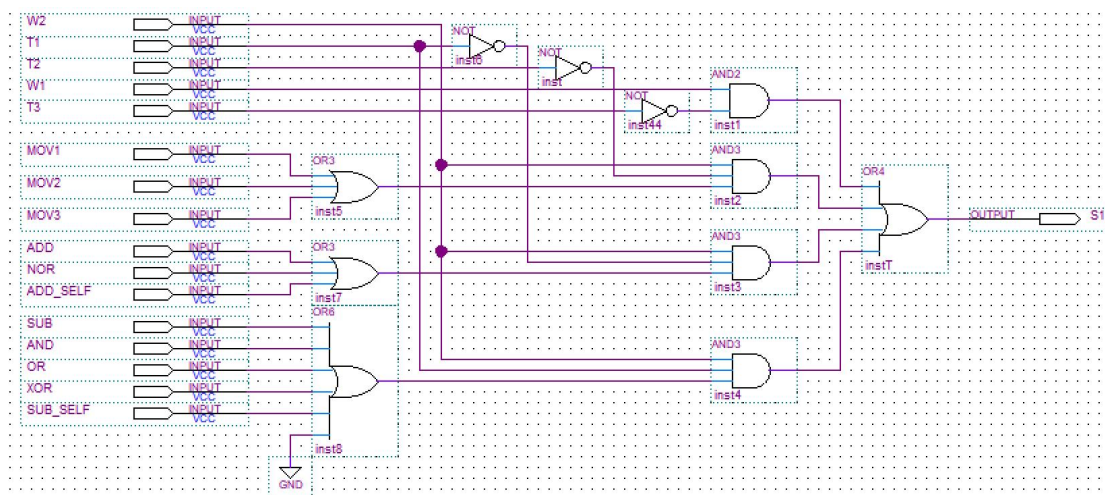
(1) 加法器控制信号 M



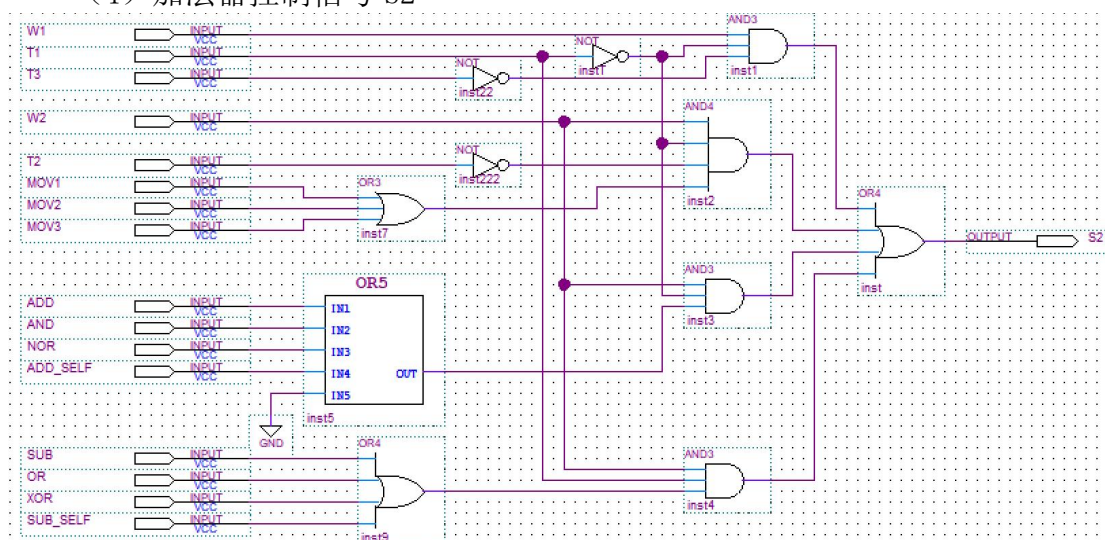
(2) 加法器控制信号 S0



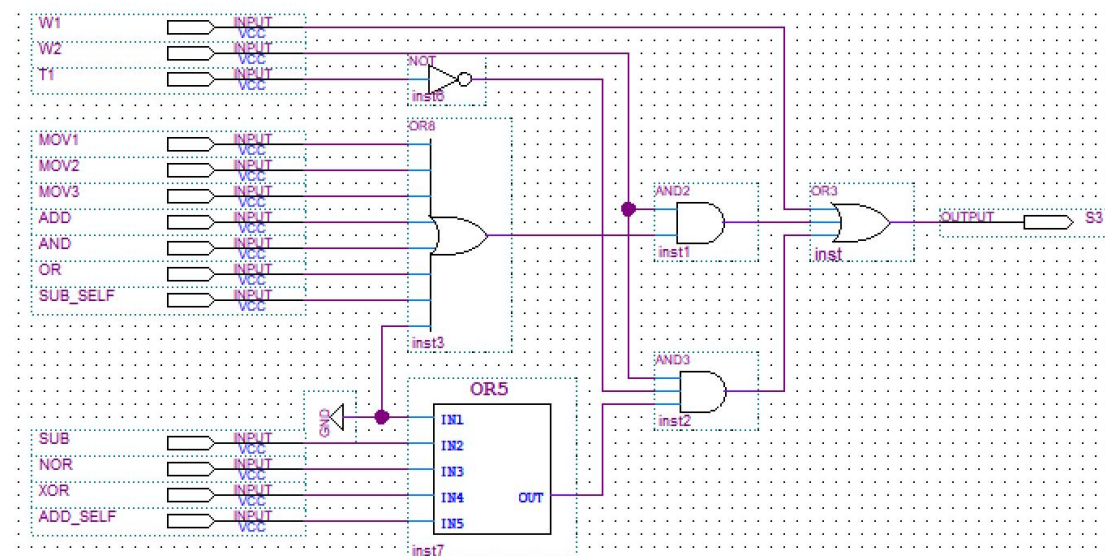
(3) 加法器控制信号 S1



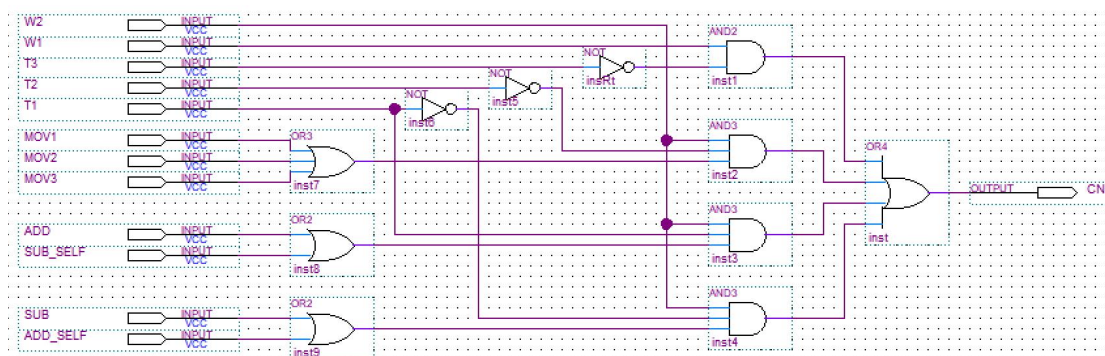
(4) 加法器控制信号 S2



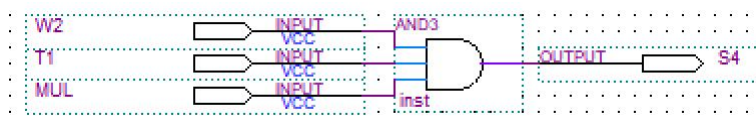
(5) 加法器控制信号 S3



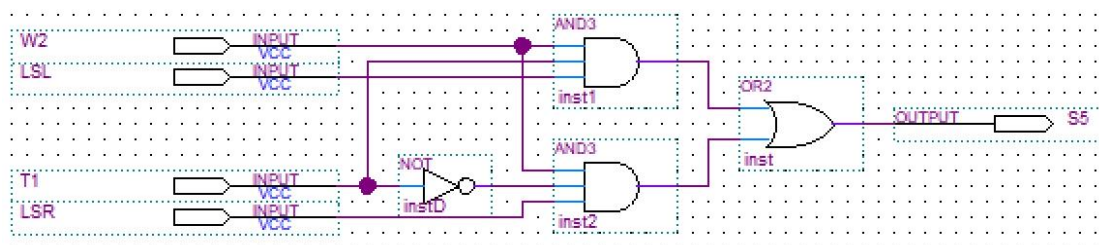
(6) 加法器控制信号 CN



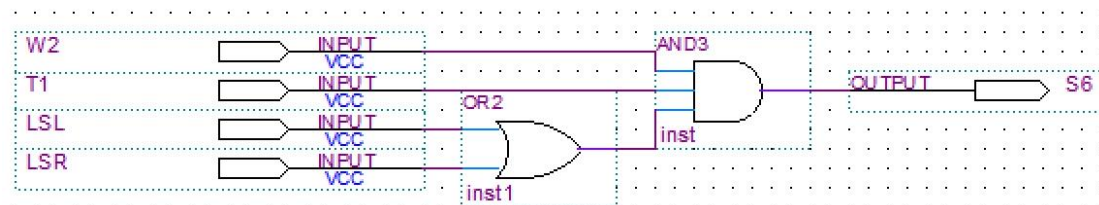
(7) 乘法器控制信号 S4



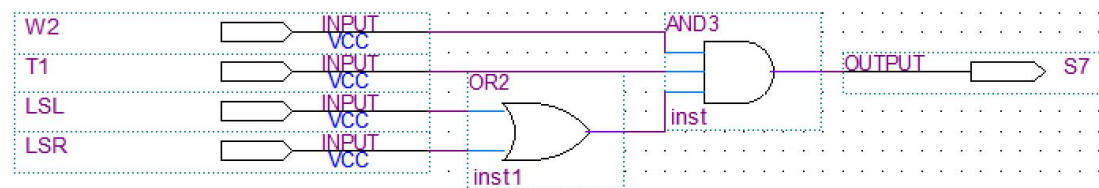
(8) 移位选择控制信号 S5



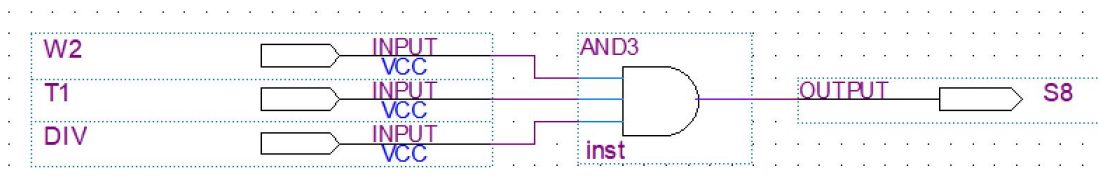
(9) 移位寄存器选择控制信号 S6



(10) 移位器控制信号 S7



(11) 除法器控制信号 S8



5.3 指令执行流程

指令执行流程（每条指令取指周期、执行周期的微操作和节拍安排）如下：

取指周期	T1	PC->MAR														
	T2	RAM->IR														
	T3	PC+1->PC														
指令		MOV1	MOV2	ADD	MOV3	SUB	AND	OR	NOR	XOR	INC	DEC	SHL	SHR	DIV	HLT
执行周期	T1	PC->MAR	PC->MAR	R0+R1->R0	PC->MAR	R0-R1->R0	R0&R1->R0	R0 R1->R0	R0 NOR R1->R0	R0@R1->R0	R0+1->R0	R0-1->R0	R0<<1->R0	R0>>1->R0	R0/R1->R0	G=1
	T2	PC+1->PC	PC+1->PC		PC+1->PC											
	T3	RAM->R0	RAM->R1		RAM->MAR											
	T4				R1->RAM											

5.4 控制信号

5.4.1 控制信号列表

根据前面的指令执行流程设置的控制信号列表如下：

状态	W1 (取指周期)	W2 (执行周期)														
指令	所有指令	MOV1	MOV2	ADD	MOV3	SUB	MUL	AND	OR	NOR	XOR	INC	DEC	SHL	SHR	DIV/HLT
MA (选择RAM)	T2	T3			T3											
RA (选择R0)				T1	T4	T1	T1	T1	T1	T1	T1	T1	T1	T1	T1	
PB (选择PC)	T1、T3	T1、T2	T1、T2		T1、T2											
RB (选择R1)				T1		T1	T1	T1	T1	T1	T1					T1
CPR0 (R0脉冲)		T3		T1		T1	T1	T1	T1	T1	T1	T1	T1	T1	T1	T1
CPR1 (R1脉冲)			T3													
CRPC (PC脉冲)	T3	T2	T2		T2											
CPIR (IR脉冲)	T2															
CPMAR (MAR脉冲)	T1	T1	T1		T1、T3											
RD (读信号)	T2	T3	T3		T3											
WD (写信号)					T4											
G (停机)																T1
M (加法器控制)	1(T1)、1(T2)、0(T3)	1(T1)、0(T2)、1(T3)	1(T1)、0(T2)、1(T3)	0(T1)	1(T1)、0(T2)、1(T3)、1(T4)	0(T1)		1(T1)	1(T1)	1(T1)	1(T1)	0(T1)	0(T1)			
S0 (加法器控制)	0(T1)、1(T2)、1(T3)	0(T1)、1(T2)、1(T3)	0(T1)、1(T2)、1(T3)	1(T1)	0(T1)、1(T2)、1(T3)、1(T4)	0(T1)		1(T1)	0(T1)	1(T1)	0(T1)	0(T1)	1(T1)			
S1 (加法器控制)	1(T1)、1(T2)、0(T3)	1(T1)、0(T2)、1(T3)	1(T1)、0(T2)、1(T3)	0(T1)	1(T1)、0(T2)、1(T3)、1(T4)	1(T1)		1(T1)	1(T1)	0(T1)	1(T1)	0(T1)	1(T1)			
S2 (加法器控制)	0(T1)、1(T2)、0(T3)	0(T1)、0(T2)、1(T3)	0(T1)、0(T2)、1(T3)	0(T1)	0(T1)、0(T2)、1(T3)、1(T4)	1(T1)		0(T1)	1(T1)	0(T1)	1(T1)	0(T1)	1(T1)			
S3 (加法器控制)	1(T1)、1(T2)、1(T3)	1(T1)、1(T2)、1(T3)	1(T1)、1(T2)、1(T3)	1(T1)	1(T1)、1(T2)、1(T3)、1(T4)	0(T1)		1(T1)	1(T1)	0(T1)	0(T1)	0(T1)	1(T1)			
CN (加法器控制)	0(T3)	0(T2)	0(T2)	1(T1)	0(T2)	0(T1)								0(T1)	1(T1)	
S4 (控制乘法)							T1									
S5 (移位方向选择)														1(T1)	0(T1)	
S6 (移位寄存器选择)														1(T1)	1(T1)	
S7 (控制移位)														1(T1)	1(T1)	
S8 (控制除法)																T1

5.4.2 控制信号的逻辑表达式

根据控制信号列表得到的控制信号的逻辑表达式如下，并该逻辑表达式恰好可以与 5.2.4 的时序控制信号形成部件相匹配：

(一) 选择信号：

(1) RAM 的 A 选择信号 MA

$$MA = W_1T_2 + W_1T_3(MOV1 + MOV2 + MOV3)$$

(2) R0 的 A 选择信号 RA

$$RA = W_2[T_1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR + INC + DEC + SHL + SHR) + T_4MOV3]$$

(3) PC 的 B 选择信号 PB

$$PB = W_1(T_1 + T_3) + W_2(T_1 + T_2)(MOV1 + MOV2 + MOV3)$$

(4) R1 的 B 选择信号 RB

$$RB = W_2T_1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR)$$

(二) 激活信号

(1) R0 的激活信号 CPR0

$$CPR0 = W_2[T_3MOV1 + T_1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR + INC + DEC + SHL + SHR)]$$

(2) R1 的激活信号 CPR1

$$CPR1 = W_2T_3MOV2$$

(3) PC 的激活信号 CPPC

$$CPPC = W_1T_3 + W_2T_2(MOV1 + MOV2 + MOV3)$$

(4) IR 的激活信号 CPIR

$$CPIR = W_1T_2$$

(5) MAR 的激活信号 CPMAR

$$CPMAR = W_1T_2 + W_2[T_1(MOV1 + MOV2 + MOV3) + T_3MOV3]$$

(6) RAM 读信号 RD

$$RD = W_1T_2 + W_2T_3(MOV1 + MOV2 + MOV3)$$

(7) RAM 写信号 WR

$$WR = W_2T_4MOV3$$

(三) 停机信号

$$G = W_2T_1HLT$$

(四) 运算控制信号

(1) 加法器控制信号 M

$$M = W_1\overline{T_3} + W_2\overline{T_2}(MOV1 + MOV2 + MOV3) + W_2\overline{T_1}(ADD + SUB + INC + DEC) + W_2T_1(AND + OR + NOR + XOR)$$

(2) 加法器控制信号 S0

$$S0 = W_1 \overline{T_1} + W_2 \overline{T_1} (\text{MOV1} + \text{MOV2} + \text{MOV3} + \text{SUB} + \text{OR} + \text{XOR} \\ + \text{INC}) + W_2 T_1 (\text{ADD} + \text{AND} + \text{NOR} + \text{DEC})$$

(3) 加法器控制信号 S1

$$S1 = W_1 \overline{T_3} + W_2 \overline{T_2} (\text{MOV1} + \text{MOV2} + \text{MOV3}) + W_2 \overline{T_1} (\text{ADD} + \text{NOR} \\ + \text{INC}) + W_2 T_1 (\text{SUB} + \text{AND} + \text{OR} + \text{XOR} + \text{DEC})$$

(4) 加法器控制信号 S2

$$S2 = W_1 \overline{T_1} \overline{T_3} + W_2 \overline{T_1} \overline{T_2} (\text{MOV1} + \text{MOV2} + \text{MOV3}) + W_2 \overline{T_1} (\text{ADD} \\ + \text{AND} + \text{NOR} + \text{INC}) + W_2 T_1 (\text{SUB} + \text{OR} + \text{XOR} + \text{DEC})$$

(5) 加法器控制信号 S3

$$S3 = W_1 + W_2 (\text{MOV1} + \text{MOV2} + \text{MOV3} + \text{ADD} + \text{AND} + \text{OR} + \text{DEC}) \\ + W_2 \overline{T_1} (\text{SUB} + \text{NOR} + \text{XOR} + \text{INC})$$

(6) 加法器控制信号 CN

$$\text{CN} = W_1 \overline{T_3} + W_2 \overline{T_2} (\text{MOV1} + \text{MOV2} + \text{MOV3}) + W_2 \overline{T_2} (\text{SUB} + \text{INC}) \\ + W_2 T_1 (\text{ADD} + \text{DEC})$$

(7) 乘法器控制信号 S4

$$S4 = W_2 T_1 \text{MUL}$$

(8) 移位选择控制信号 S5

$$S5 = W_2 \overline{T_1} \text{SHR} + W_2 T_1 \text{SHL}$$

(9) 移位寄存器选择控制信号 S6

$$S6 = W_2 T_1 (\text{SHR} + \text{SHL})$$

(10) 移位器控制信号 S7

$$S7 = W_2 T_1 (\text{SHR} + \text{SHL})$$

(11) 除法器控制信号 S8

$$S8 = W_2 T_1 \text{DIV}$$

根据上面的逻辑表达式，利用简单的与门、或门和非门，就可以实现的对应的控制信号的生成，并且将对应的信号送到对应的输入即可实现硬布线实现的CU与模型机主体的连接，并且形成完整的通路。具体的时序控制信号形成部件的实现见 5.2.4。基于硬布线实现的模型机的原理图见 4.6。

5.5 RAM 应用程序设计

这里给出一个指令的混合测试，用于测试所有设计指令的正确与否。

Complex testing of instructions

```

1:  MOV1    R0,5      ;set r0=5
2:  MOV2    R1,2      ;set r1=2
3:  ADD     R0,R1     ;r0+r1->r0
4:  MOV3    (FF),R0   ;send r0 to ram, display the ans
5:  SUB     R0,R1     ;r0-r1->r0
6:  MOV3    (FE),R0   ;send r0 to ram, display the ans
7:  MUL     R0,R1     ;r0*r1->r0
8:  MOV3    (FD),R0   ;send r0 to ram, display the ans
9:  AND     R0,R1     ;r0&r1->r0
10: MOV3    (FC),R0   ;send r0 to ram, display the ans
11: OR      R0,R1     ;r0|r1->r0
12: MOV3    (FB),R0   ;send r0 to ram, display the ans
13: NOR     R0,R1     ;r0 nor r1->r0
14: MOV3    (FA),R0   ;send r0 to ram, display the ans
15: XOR     R0,R1     ;r0 xor r1->r0
16: MOV3    (F9),R0   ;send r0 to ram, display the ans
17: INC     R0        ;r0+1->r0
18: MOV3    (F8),R0   ;send r0 to ram, display the ans
19: DEC     R0        ;r0-1->r0
20: MOV3    (F7),R0   ;send r0 to ram, display the ans
21: SHL     R0        ;r0<<1->r0
22: MOV3    (F6),R0   ;send r0 to ram, display the ans
23: SHR     R0        ;r0>>1->r0
24: MOV3    (F5),R0   ;send r0 to ram, display the ans
25: DIV     R0,R1     ;r0/r1->r0
26: MOV3    (F4),R0   ;send r0 to ram, display the ans
27: HLT                ;stop
    
```

上面的代码先给 R0 和 R1 赋予初值，然后再逐条执行全部的指令（包括了相加、相减、相乘、求与、求或、求异或、求或非、自增、自减、左移、右移和相除）。

机器语言和理论输出如下：

0x00	00000000	MOV1 R0,5	00000000	0x00
0x01	00000001	/	00000101	0x05
0x02	00000010	MOV2 R1,2	00010000	0x10
0x03	00000011	/	00000010	0x02
0x04	00000100	ADD R0,R1	00100000	0x20
0x05	00000101	MOV3 (FF),R0	00110000	0x30
0x06	00000110	/	11111111	0xFF
0x07	00000111	SUB R0,R1	01000000	0x40
0x08	00001000	MOV3 (FE),R0	00110000	0x30
0x09	00001001	/	11111110	0xFE
0x0A	00001010	MUL R0,R1	01010000	0x50
0x0B	00001011	MOV3 (FD),R0	00110000	0x30
0x0C	00001100	/	11111101	0xFD
0x0D	00001101	AND R0,R1	01100000	0x60
0x0E	00001110	MOV3 (FC),R0	00110000	0x30
0x0F	00001111	/	11111100	0xFC
0x10	00010000	OR R0,R1	01110000	0x70
0x11	00010001	MOV3 (FB),R0	00110000	0x30
0x12	00010010	/	11111011	0xFB
0x13	00010011	NOR R0,R1	10000000	0x80
0x14	00010100	MOV3 (FA),R0	00110000	0x30
0x15	00010101	/	11111010	0xFA
0x16	00010110	XOR R0,R1	10010000	0x90
0x17	00010111	MOV3 (F9),R0	00110000	0x30
0x18	00011000	/	11111001	0xF9
0x19	00011001	INC R0	10100000	0xA0
0x1A	00011010	MOV3 (F8),R0	00110000	0x30
0x1B	00011011	/	11111000	0xF8
0x1C	00011100	DEC R0	10110000	0xB0
0x1D	00011101	MOV3 (F7),R0	00110000	0x30
0x1E	00011110	/	11111011	0xF7
0x1F	00011111	SHL R0	11000000	0xC0
0x20	00100000	MOV3 (F6),R0	00110000	0x30
0x21	00100001	/	11111010	0xF6
0x22	00100010	SHR R0	11010000	0xD0
0x23	00100011	MOV3 (F5),R0	00110000	0x30
0x24	00100100	/	11111010	0xF5
0x25	00100101	DIV R0,R1	11100000	0xE0
0x26	00100110	MOV3 (F4),R0	00110000	0x30
0x27	00100111	/	11111010	0xF4
0x28	00101000	HLT	11110000	0xF0

模拟仿真结果如下，板子上实际结果与模拟仿真结果相同（已验收）：



由于 FPGA 板子上的实验结果难以拍照记录，这里不进行展示，但是可以保证的是，上面所有的实验结果均已验收，且 FPGA 板子上的结果与模拟仿真的结果一致。

第 6 章 课程设计总结

6.1 遇到的问题和解决方案

问题 1: 在微程序控制中, 如果单条微指令在给出选择信号的同时, 也给出激活信号, 在这一时刻, 激活信号送入的数据是上一秒的结果 (即新产生的结果没有来得及送入)。

解决方案 1: 这是一个非常典型的时序问题, 在同一时刻给出多个信号, 会由于线路产生的微小延迟导致时序产生错误。一个常用的解决方案是将时钟周期分为上跳沿和下跳沿, 在上跳沿给出选择信号, 下跳沿给出激活信号。但是在实际情况中, 由于上跳沿和下跳沿已经分给了 uPC 的产生信号和 ROM 的访存信号, 所以该方法无法实现。因此这里采用了将一条微指令分成两条微指令, 第一条微指令只给选择信号, 而激活信号为 0; 第二条微指令既存在选择信号, 又存在激活信号, 这样激活信号就设置出了一个上跳沿, 能够得到正确的结果。

问题 2: 在左移和右移操作时, 采用了 74198 器件, 但是这个操作和加法器、乘法器、除法器操作不同, 后面的几个器件的结果是立即产生的, 即, 无论该器件是否选中, 只要存在输入, 那么就会有输出 (根本原因是不存在触发器)。但是移位器进行移位需要两步操作, 首先需要进行置数, 然后才能进行移位。

解决方案 2: 将移位指令分成两部分完成, 第一部分是置数操作, 即让数据打入 ALU 中 (其实是移位器或者 74198 中), 然后第二次给出计算结果。这样解决了上面提出的问题。

问题 3: 运算结果与预期存在误差应该如何解决?

解决方案 3: 首先如果模拟仿真结果出现错误, 应该通过在相关输出 (uPC、ROM、RAM、ALU 等), 来检查能否正确生成下地址、能否正确取出微指令、能否正确取出指令、能否正确选择信号、能否正确产生计算结果、能否正确将数据打入等。如果模拟仿真结果正确, 但是上板后结果错误, 这时应该考虑是否是时序产生错误。优先通过手动产生信号的方式进行检查, 如果手动产生 CLK 结果正确, 但是方波信号结果错误, 考虑是器件电路问题, 可以尝试通过添加输出、或者减少输出的方式, 有可能会取得正确的结果。

问题 4: 指令条数不够, 或者单条指令如何执行不同的寻址方式?

解决方案 4: 在通常情况下, 单条指令只能执行一种寻址方式, 比如在课件中, MOV 指令设置了很多 OP, 但是在实际情况中, MOV 指令只存在一种 OP, 需要经过寻址方式的判断, 来计算下一步应该如何执行。上面所说的方式为条件分支或者分支转移。实际的解决方案是, 在进入程序入口地址之后, 先按照原操作数的寻址方式进行一次跳转, 然后执行对应的代码, 然后再根据目的操作数的寻址方式进行二次跳转, 这样就可以利用一条指令, 执行多种寻址方式的指令。

6.2 设计的优点与缺点

6.2.1 设计的优点

在该模型机中，我们将指令和数据全部拓展到 16 位，允许执行 16 位的数据运算。还设计了 31 个指令组，共允许执行 80 条指令。并且模型机的功能较为完善，不仅能够实现加减基础运算，更额外设计了与、或、非、异或、求负、逻辑左移、逻辑右移等逻辑运算，以及乘法、除法、自增、自减、算数左移、算数右移等算数运算。不仅如此，还额外设置了多种跳转方式，如无条件跳转、标志位为零则跳转、标志位不为零则跳转，并且基于上面的跳转实现了 loop 指令，能够实现 do-while 循环、while 循环和 for 循环。除了跳转方式外，还基于内存开设了一个栈空间，可以执行压栈、弹栈等栈操作，还基于栈实现了子程序调用指令和子程序返回指令，并且通过间接寻址、变址寻址等方式可以访问栈空间的内容，进而实现利用栈传参进行子程序的调用。除了上面所说，我们还基于 8086 实现了部分其他功能指令，比如 CMP 比较指令、XCHG 交换指令等。在寻址方式上，我们也进行了相对的拓展，能够支持立即数寻址、直接寻址、间接寻址、寄存器寻址、PC 相对寻址、变址寻址等多种寻址。

为了压缩指令，还采用了条件分支的方法，在相同操作码中可以支持多种寻址方式，即在进入程序入口地址后，对操作数再进行一次判断，从而跳转到该 OP 的对应操作码的微程序段中，进而对操作码进行了很大的压缩，我们仅仅使用了 5 位操作码就可以执行 80 条指令，而对于普通的方式则至少需要 7 位操作码。

可以看到即使我们实现了多种功能，但是模型机的时序逻辑非常清晰，在 5 种不同的功能的测试中，均没有产生竞争冒险问题和其他的时序逻辑问题，结果准确高效，并且系统非常稳定。

6.2.2 设计的缺点

本系统由于没有双字长指令，因此虽然采用的是 16 位指令，但是由于受到操作数的影响，操作码只能够设置 5 位，这就进一步导致指令条数不方便进一步的扩展。可以考虑使用双字长指令来增大操作码的位数，因此设计更多的指令。

关于硬布线实现的模型机也存在一定的问题和不足。因为硬布线缺少微指令的灵活性，因此难以实现分支转移，这就导致很多能够通过微指令完成的指令，在硬布线上难以实现。除此之外，还可以通过采用流水线的方式来提高系统的执行效率，通过建立 cache 减少访存的次数等。

另外，还有一个很明显的不足是，由于寄存器设置的较少，很多本身应该存在于 CPU 内部的寄存器，被迫转移到了内存中，比如 CX 寄存器、SP 寄存器等，这样就进一步导致系统执行效率的下降。或者本身应该是两个寄存器的功能，却被我们放在一个寄存器中完成了，比如说 R3 寄存器兼备条件寄存器的功能。使得程序编写的难度提高。

并且寄存器设置较少，也就进一步导致很多数据无法存放在寄存器中，需要存储在内存中，这就进一步导致访存的次数增多，进而导致系统执行效率变低。

6.3 收获与体会

通过本次模型机的设计，我深刻地感受到计算机系统的构建不仅仅是理论的堆砌，更是一个涉及到各个层面、各类硬件与控制机制相互配合的复杂过程。在实验过程中，模拟和实现了计算机的基本功能模块，包括控制器、运算器、存储器等，尤其是通过实现微程序控制器与硬布线控制器两种控制方式，让我更加深入理解了计算机系统的设计原理。

在微程序控制器与硬布线控制器实现的两个实验的对比中，我对两者的优缺点有了更加清晰的认识。微程序控制器的控制功能依赖于控制器和微指令存储器的紧密配合，相比硬布线控制器，它的电路结构更为规整，指令的增加与修改也相对简单。特别是在系统需求变化时，微程序控制器的灵活性表现得尤为突出，只需修改微指令存储器中的内容即可实现新指令的扩展。然而，微程序控制器由于每条微指令都需要从存储器中读取，导致其执行速度相对较慢，尤其在要求高性能的计算环境中，这一瓶颈尤为明显。

与此相对的是硬布线实现的控制结构，它通过逻辑门电路直接实现控制功能，设计较为复杂，但在执行速度上具有显著优势。硬布线控制器的设计虽然繁琐，但其硬件直接实现的控制信号可以提供极高的执行效率，这对于高速运算尤为重要。比如，在现代的高性能计算机中，为了追求更快的运算速度，许多核心部件往往采用硬布线控制逻辑，这一设计思想尤其在 RISC 架构中得到了广泛应用。这让我更加明白了不同控制方式在不同应用场景下的优势和局限。

此外，这次实验让我深刻体会到了硬件设计与理论知识之间的密切联系和计算机组成与设计课程的重要性。在实践中，我不仅仅是将之前学到的数字逻辑、电子电路、计算机系统原理等理论应用到实际的电路设计中，还通过每一步的调试和优化，进一步加深了对这些知识的理解。尤其是当模型机的各个模块逐渐搭建完成时，我能清晰地看到理论和实践之间的紧密联系，这不仅加深了我对计算机系统架构的理解，还让我对计算机性能优化有了更加直观的认识。

总的来说，这次实验不仅让我获得了宝贵的实践经验，还极大地提升了我的问题解决能力。在经历了从理论到实践的过程后，我对计算机的工作原理有了更加全面的认识。我意识到，计算机系统的设计不仅仅是技术问题，更是一个涉及到工程、优化与决策的复杂过程。这次实验为我今后的学习和发展打下了坚实的基础，同时也激发了我对计算机硬件与系统架构设计更深层次的兴趣。

附录

附录 1：数据通路图

附录 2：微程序流程图

附录 3：微程序

附录 4：各控制信号的列表

附录 5：各控制信号的逻辑表达式